CareBridge AI: An AI-Assisted, Multi-Tenant Hospital Operations and Clinical Coordination Platform

SWE3004 – Software Design and Development Project

Submitted in partial fulfillment of the requirements for the degree of

Master of Technology in Software Engineering

in

Department of Software and Systems Engineering

by

[Student Name]

[Registration Number]

Under the guidance of

[Off-Campus Internship Guide Name, Designation, Company]

School of Computer Science Engineering and Information Systems

VIT, Vellore



November, 2026

CareBridge AI: An AI-Assisted, Multi-Tenant Hospital Operations and Clinical Coordination Platform

SWE3004 – Software Design and Development Project

Submitted in partial fulfillment of the requirements for the degree of

Master of Technology in Software Engineering

in

Department of Software and Systems Engineering

by

[Student Name]

[Registration Number]

Under the guidance of

[Off-Campus Internship Guide Name, Designation, Company]

School of Computer Science Engineering and Information Systems

VIT, Vellore



November, 2026

DECLARATION

I hereby declare that the SWE3004 - Software Design and Development Project entitled "CareBridge AI: An AI-Assisted, Multi-Tenant Hospital Operations and Clinical Coordination Platform" submitted by me, for the award of the degree of Master of Technology in Software Engineering in the Department of Software and Systems Engineering, School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, is a record of bonafide work carried out by me under the supervision of [Off-Campus Internship Guide Name, Designation, Company] and [VIT Faculty Guide Name, Designation, SCORE, VIT, Vellore].

I further declare that the work reported in this project / internship has not been submitted and will not be submitted, either in part or in full, for the award of any other degree ordiploma in this institute or any other institute or university.

Place: Vellore

Date:

Signature of the Candidate

CERTIFICATE

This is to certify that the SWE3004 - Software Design and Development Project entitled "CareBridge AI: An AI-Assisted, Multi-Tenant Hospital Operations and Clinical Coordination Platform" submitted by [Student Name] & [Registration Number], SCORE, VIT, for the award of the degree of Master of Technology in Software Engineering in the Department of Software and Systems Engineering, is a record of bonafide work carried out by him / her under my supervision during the period 06.07.2026 to 03.11.2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute orany other institute or university. The project / internship fulfills the requirements and regulations ofthe University and in my opinion meets the necessary standards for submission.

Place: Vellore
Date:

Signature of the Company Guide
<For off-campus internship students only>

Signature of the VIT-SCORE - Guide

Internal Examiner **External Examiner**

Head of the Department
Department of Software and Systems Engineering

Off-Campus Internship Certificate

<For off-campus internship students only>

ABC COMPANY LIMITED

Office Address: 34/ 67 Rani Nagar, Delhi- 89

Phone No: xxxxxxxxx Email: abc@gmail.com Website: www.abcdef.com

Date: DD/MM/YYYY

TO WHOM IT MAY CONCERN

This is to certify that (Ms/Mr/Mrs. _____, student of _____, has successfully completed a summer internship in the field of _____ from ___ to ___ under guidance of _____.

During the period of her/ his internship program with us, she/ he had been exposed to different processes and was found diligent, hardworking and inquisitive.

We wish her/ him every success in her/his life and career.

For ABC Company Ltd.

Authorised Signature

_____

ACKNOWLEDGEMENT

It is my pleasure to express a deep sense of gratitude to my Project / Internship guide [Dr. / Prof. Guide Name, Designation], School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore, for constant guidance and continual encouragement throughout this endeavour. My association with my guide is not confined to academics only; it has been a great opportunity to work with an expert in the field of AI-assisted healthcare software engineering and cyber-physical security for the Internet of Medical Things.

"I would like to express my heartfelt gratitude to Honorable Chancellor **Dr. G Viswanathan**; respected Vice Presidents **Dr. Sankar Viswanathan**, **Dr. Sekar Viswanathan**, Vice Chancellor **Dr. V. S. Kanchana Bhaaskaran**; Pro-Vice Chancellor **Dr. Partha Sharathi Mallick**; and Registrar **Dr. Jayabarathi T**.

My whole-hearted thanks to Dean **Dr. Daphne Lopez**, School of Computer Science Engineering and Information Systems, Head, Department of Software and Systems Engineering **Dr. Thanapal P**, MTech - Software Engineering Project Coordinators **Dr. Thilagavathi M and Dr. Brijendra Singh**, SCORE School Project Coordinator **Dr. Karthikeyan P,** all faculty, staff and members working as limbs of our university for their continuous guidance throughout my course of study in unlimited ways.

It is indeed a pleasure to thank my parents and friends who persuaded and encouraged me to take up and complete my **project/ Internship** successfully. Last, but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly towards the successful completion of the **project/ Internship**.

Place: Vellore
Date: [Student Name]

Executive Summary

CareBridge AI is a safety-first, multi-tenant hospital operations and clinical coordination platform. It unifies patient administration, appointment scheduling, caregiver linkage, clinician-verified document intake, allergy and drug-interaction conflict alerting, prioritised care tasks, tamper-evident prescriptions with adherence tracking, deterministic insurance estimation, and cached operations analytics on a single tenant-isolated data model. A conversational assistant grounded in the hospital's own APIs provides navigation and operations support, while an optional voice prototype remains disabled unless configured. Every order-sensitive function is advisory and human-in-the-loop. The project was delivered through a fifteen-phase plan (Phases 0 to 14) with strict phase-gate governance. Phases 0 to 11 are reported complete; a proposed Phase 12 would add a physiology-aware cross-sensor consistency model (CSCM) for detecting coordinated multi-channel false data injection attacks in the Internet of Medical Things. Phases 13 and 14 (hardening and release) remain pending. This report documents the verified architecture, the evidence-based status of each phase, the evaluated CSCM research, and the work remaining before submission.

CONTENTS

CHAPTER 1 INTRODUCTION 1
1.1 Background of the Domain 1
1.2 Problem Statement 2
1.3 Motivation 3
1.4 Objectives of the Project 3
1.5 Research Questions 4
1.6 Scope of the Project 5
1.7 Contributions 5
1.8 Organisation of the Report 6
CHAPTER 2 LITERATURE REVIEW 8
2.1 Introduction 8
2.2 Hospital Information Systems, EHRs and Multi-Tenancy 8
2.3 Conversational AI and Large Language Model Agents in Healthcare 9
2.4 Document Intelligence in Clinical Settings 10
2.5 Clinical Decision Support: Allergy and Drug-Interaction Alerting 10
2.6 Prescription Integrity and Medication Adherence 11
2.7 Insurance Coordination and Financial Estimation 11
2.8 Healthcare Operations Analytics 12
2.9 IoMT Security and False Data Injection Attacks 12
2.10 Time-Series and Graph-Based Anomaly Detection 13
2.11 Research Gap 13
2.12 Summary 14
CHAPTER 3 REQUIREMENT ANALYSIS 15
3.1 Introduction 15
3.2 Requirement Elicitation Approach 15
3.3 Stakeholders and Roles 15
3.4 Functional Requirements 16
3.5 Non-Functional Requirements 17
3.6 Use Cases 18
3.7 Technology Stack 19
3.8 Constraints and Assumptions 19
3.9 Requirements Traceability Matrix 20
3.10 Requirement Prioritisation 20
3.11 Summary 21
CHAPTER 4 SYSTEM DESIGN 22
4.1 Introduction 22
4.2 Design Principles 22
4.3 High-Level Architecture 23
4.4 Deployment View 24
4.5 Data Model 25

4.6 API Design 27
4.7 Authentication and Authorisation Flow 29
4.8 Detailed Interaction Designs 30
4.9 Security and Threat Model 34
4.10 Privacy and Compliance Considerations 35
4.11 Proposed CSCM Integration Design 35
4.12 Phase Dependency Design 36
4.13 Summary 37
CHAPTER 5 IMPLEMENTATION 38
5.1 Introduction and Delivery Approach 38
5.2 Phase 0: Audit and Baseline 38
5.3 Phase 1: Foundation 38
5.4 Phase 2: Database, Authentication and Row-Level Security 39
5.5 Phase 3: Core Workflow Services 39
5.6 Phase 4: Front-End Application 40
5.7 Phase 5: Conversational Clinical and Operations Agent 40
5.8 Phase 6: Document Intelligence 40
5.9 Phase 7: Clinical Conflict Detection 41
5.10 Phase 8: Care Task Management 41
5.11 Phase 9: Prescription Integrity and Adherence 41
5.12 Phase 10: Conversational Assistant and Optional Voice 42
5.13 Phase 11: Insurance Estimation and Operations Analytics 42
5.14 Phase 12: CSCM-IoMT Integration (Proposed) 43
5.15 Phase 13: Hardening (Pending) 43
5.16 Phase 14: Release (Pending) 43
5.17 Phase Status Summary 43
5.18 Cross-Cutting Implementation Concerns 44
5.19 Summary 45
CHAPTER 6 ALGORITHMS AND PSEUDOCODE 46
6.1 Introduction 46
6.2 Notational Conventions 46
6.3 Tenant-Scoped Authorisation 46
6.4 Double-Booking Prevention 47
6.5 Document Extraction and Clinician Verification 47
6.6 Clinical Conflict Detection 48
6.7 Prescription Signing and Verification 48
6.8 Insurance Estimation 49
6.9 Cached Analytics 49
6.10 CSCM Problem Formulation 49
6.11 CSCM Training and Evaluation Algorithm 50
6.12 Complexity Analysis 51

6.13 Summary....52
CHAPTER 7 TESTING AND EVALUATION....53
7.1 Introduction....53
7.2 Testing Strategy....53
7.3 Test Environment....53
7.4 Backend Test Suites....54
7.5 Front-End Static and Build Verification....54
7.6 Security Testing....54
7.7 Research Evaluation Methodology....55
7.8 Evidence Register....55
7.9 Verification Gaps and Limitations....56
7.10 Test Case Catalogue....56
7.11 Summary....58
CHAPTER 8 RESULTS AND DISCUSSION....59
8.1 Introduction....59
8.2 Platform Results....59
8.3 CSCM Absolute Performance....60
8.4 Architectural Ablation....63
8.5 Statistical Robustness....63
8.6 Error and Failure Analysis....64
8.7 Discussion....66
8.8 Threats to Validity....67
8.9 Summary....67
CHAPTER 9 PROJECT MANAGEMENT....68
9.1 Introduction....68
9.2 Delivery Methodology....68
9.3 Schedule and Milestones....68
9.4 Work Breakdown Structure....69
9.5 Effort Estimation....69
9.6 Cost Analysis....70
9.7 Risk Management....71
9.8 Governance and Phase Gates....71
9.9 Quality Management....72
9.10 Assumptions, Constraints and Dependencies....72
9.11 Lessons Learned....73
9.12 Summary....73
CHAPTER 10 CONCLUSION AND FUTURE WORK....74
10.1 Summary of the Work....74
10.2 Answers to the Research Questions....74
10.3 Contributions Revisited....75
10.4 Limitations....75

10.5 Future Work....76
10.6 Concluding Remarks....76
REFERENCES....78
APPENDICES....80
APPENDIX A: Sample Source Code....80
APPENDIX B: Configuration and Setup....82
APPENDIX C: Database Migration Inventory....83
APPENDIX D: Test Commands and Reported Results....83
APPENDIX E: Submission-Readiness Checklist....84
APPENDIX F: Phase-to-Chapter Traceability....85

List of Figures

Fig. 4.1 System context of CareBridge AI showing actors, the platform and external services 23
Fig. 4.2 Component architecture of CareBridge AI 24
Fig. 4.3 Deployment view of the development and hosted configurations 25
Fig. 4.4 Entity-relationship overview of the tenant-scoped schema 26
Fig. 4.5 Authentication and authorisation flow from token verification to row-level security 29
Fig. 4.6 Sequence diagram for appointment booking with double-booking prevention 30
Fig. 4.7 Document extraction and clinician verification flow 31
Fig. 4.8 Clinical conflict detection and acknowledgement flow 32
Fig. 4.9 Prescription signing and public verification flow 33
Fig. 4.10 Deterministic insurance coverage estimation flow 33
Fig. 4.11 Operations analytics aggregation and caching flow 34
Fig. 4.12 Proposed CSCM-IoMT inference flow (Phase 12, not implemented) 36
Fig. 4.13 Phase dependency diagram for Phases 0 to 14 37
Fig. 8.1 Absolute performance of the three models on both datasets 60
Fig. 8.2 Architecture of the evaluated multi-channel models 63
Fig. 8.3 Bootstrap confidence intervals for the architectural differences 64
Fig. 8.4 Synthetic attack generation showing additive, multiplicative and drift patterns 65
Fig. 8.5 Per-sample attack mask used to derive window-level labels 66

List of Tables

Table 1.1 Contribution, phase and verification status....5
Table 2.1 Representative large language model and conversational studies and their relevance....10
Table 2.2 Families of anomaly detection and their applicability to IoMT FDIA....13
Table 3.1 Stakeholder roles and their primary concerns....15
Table 3.2 Functional requirements....16
Table 3.3 Non-functional requirements....17
Table 3.4 Principal use cases....18
Table 3.5 Technology stack....19
Table 3.6 Requirements traceability matrix....20
Table 3.7 Requirement prioritisation....21
Table 4.1 Database tables and their isolation mechanisms....26
Table 4.2 API endpoint catalogue....27
Table 4.3 Threat model, mitigations and residual risk....34
Table 5.1 Tables created in Phase 2 and their key constraints....39
Table 5.2 Phase 11 reported verification figures....42
Table 5.3 Phase status, evidence and verification label....43
Table 6.1 Complexity of the principal algorithms....51
Table 7.1 Reported backend test outcomes....54
Table 7.2 Security checks and their basis....55
Table 7.3 Evidence register....56
Table 7.4 Representative test case catalogue....57
Table 8.1 Platform result summary....59
Table 8.2 Absolute performance of the three models across both datasets (mean plus or minus standard deviation over three seeds)....60
Table 8.3 Architectural differences averaged over three seeds....63
Table 8.4 Statistical assessment of the architectural differences (selected seed-level results)....63
Table 9.1 Phase schedule and milestones....68
Table 9.2 Work breakdown structure....69
Table 9.3 Approximate effort apportionment....70
Table 9.4 Cost analysis for the project....70
Table 9.5 Risk register....71
Table 9.6 Dependencies and fallbacks....72
Table 0.1 Backend environment variables by purpose....83
Table 0.2 Database migration inventory....83
Table 0.3 Test commands and reported outcomes....84
Table 0.4 Submission-readiness checklist....84
Table 0.5 Phase-to-chapter traceability....85

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
APA	American Psychological Association
CDSS	Clinical Decision Support System
CSCM	Cross-Sensor Consistency Model
CSV	Comma-Separated Values
DDI	Drug-Drug Interaction
DPDP	Digital Personal Data Protection (Act, 2023, India)
ER	Entity Relationship
FDIA	False Data Injection Attack
FHIR	Fast Healthcare Interoperability Resources
GNN	Graph Neural Network
GRU	Gated Recurrent Unit
HMAC	Hash-based Message Authentication Code
HIPAA	Health Insurance Portability and Accountability Act
HTTPS	Hypertext Transfer Protocol Secure
ICU	Intensive Care Unit
IoMT	Internet of Medical Things
JWT	JSON Web Token
LLM	Large Language Model
MRN	Medical Record Number
NFR	Non-Functional Requirement
OCR	Optical Character Recognition
OWASP	Open Worldwide Application Security Project
PII	Personally Identifiable Information
PPG	Photoplethysmography
RBAC	Role-Based Access Control
REST	Representational State Transfer
RLS	Row-Level Security
RNN	Recurrent Neural Network
SDK	Software Development Kit
SQL	Structured Query Language
TOC	Table of Contents
UUID	Universally Unique Identifier
VIT	Vellore Institute of Technology
WESAD	Wearable Stress and Affect Detection (dataset)

Symbols and Notations

X	Multivariate physiological sensing window (C x T matrix)
C	Number of physiological channels (sensors)
T	Window length (number of samples)
z	Latent temporal representation of a channel or window
E	Edge set of the inter-sensor consistency graph
A	Adjacency / learned relation representation of the channel graph
f	Encoder function parameterised by learnable weights
s	Anomaly score produced by the detection head
τ	Decision threshold calibrated on validation data
L	Composite training objective
α, β	Weighting coefficients of the loss terms
λ	Regularisation coefficient
μ	Mean of a signal or feature
σ	Standard deviation of a signal or feature
$\oplus$	Element-wise summation used in graph message aggregation

CHAPTER 1

INTRODUCTION

1.1 Background of the Domain

Hospitals are among the most information-intensive organisations in modern society. A single patient episode can generate demographics, appointment schedules, consultation notes, laboratory reports, scanned referrals, prescriptions, pharmacy dispensing records, insurance claims and biomedical equipment telemetry. In large tertiary hospitals these functions are usually served by an integrated Hospital Information Management System (HIMS) or an Electronic Health Record (EHR). In mid-sized hospitals, nursing homes and single-speciality clinics, however, the same functions are frequently distributed across disconnected spreadsheets, paper registers, messaging applications and small stand-alone tools. The result is fragmented information, duplicated effort, delayed coordination and avoidable patient-safety risk.

The World Health Organization estimates that unsafe care causes a substantial global burden of disability and death, and it has identified medication safety as a priority global challenge (World Health Organization, 2017). In parallel, the past decade has seen three technological shifts that make rapid, low-cost digitisation feasible for smaller care providers. First, cloud-managed relational databases with built-in authentication and declarative row-level security make it possible to build multi-tenant software without a large platform team. Second, large language models (LLMs) with tool-calling interfaces make conversational interfaces viable for navigating the complex operations of a hospital. Third, the convergence of cheap wearables and networked medical devices under the umbrella of the Internet of Medical Things (IoMT) has created new attack surfaces that must be understood and defended.

CareBridge AI is a response to the first two shifts, and it positions itself to address the third through a clearly separated, research-grade subsystem. It is an AI-assisted, multi-tenant hospital operations and clinical coordination platform. It unifies patient administration, appointment scheduling, caregiver-to-patient linkage, document intake with clinician verification, clinical conflict alerting, prioritised care tasks, tamper-evident prescriptions with adherence tracking, deterministic insurance estimation and cached operations analytics on a single tenant-isolated data model. A conversational assistant grounded in the platform's own

APIs acts as an operational co-pilot. Throughout, the design principle is that software should assist, prioritise and coordinate, but should never silently author clinical decisions: all order-sensitive outputs are advisory and human-in-the-loop.

1.2 Problem Statement

The project addresses the following problem. A mid-sized hospital needs a single, secure, low-cost platform that consolidates routine operational and clinical coordination workflows, while preserving strict tenant isolation, role-based access, auditability and clinical safety. In addition, the hospital increasingly depends on networked physiological sensing devices, and coordinated false data injection attacks (FDIA) against multiple sensors simultaneously can defeat single-sensor plausibility checks. The platform must therefore also provide a principled path to detect coordinated cross-sensor inconsistency, without making unvalidated claims that such detection is already operational.

Concretely, the following operational weaknesses were identified during the Phase 0 audit of the project and are treated as the requirements drivers for the platform:

- Fragmented patient identity and demographics spread across paper and local files, with no single source of truth.
- Manual appointment scheduling that permits double-booking and provides no reliable cancellation policy enforcement.
- Weak caregiver linkage, whereby family members have no auditable, revocable channel to a patient's record.
- Document intake performed by photographing and filing documents without structured extraction or clinician verification.
- Allergy and drug-interaction risks that are discovered too late because no automated, advisory conflict scan exists.
- Care tasks (for example, follow-up calls, sample collection, insurance desk follow-up) tracked informally and lost.
- Prescriptions that are easy to alter on paper, with no tamper-evident verification and no adherence tracking.
- Insurance coverage estimated inconsistently and non-transparently, causing avoidable

patient grievances.

- Operational analytics that require manual counting and reporting.
- No mechanism to detect, or even reason about, coordinated manipulation of multiple physiological sensor channels.

The first nine weaknesses are addressed by Phases 0 to 11 of the project, which are reported as complete. The tenth weakness motivates the research component, formalised as Phase 12, which remains proposed.

1.3 Motivation

The motivation for CareBridge AI is threefold: clinical, operational and research. Clinically, the platform reduces the chance that a documented allergy, a duplicate therapy or a serious drug-drug interaction is missed, by surfacing advisory conflicts to the treating clinician with the underlying evidence and requiring an explicit acknowledgement. Operationally, it removes duplicated manual work in reception, nursing and the insurance desk, and it provides a measurable, cached operations dashboard for administrators. From a research perspective, the increasing reliance on IoMT devices introduces a security problem that is not solved by conventional per-sensor anomaly detection: an adversary who can influence several sensors can keep each individual channel within its marginal plausibility bounds while producing a physiologically impossible joint state. Detecting such coordinated attacks requires modelling the consistency between channels, which is the specific contribution that the proposed Phase 12 subsystem, the Cross-Sensor Consistency Model (CSCM), aims to make.

The overall research title preserved for the Phase 12 component is: Coordinated Multi-Channel False Data Injection Attack Detection Using Physiology-Aware Cross-Sensor Consistency Modeling in IoMT. This report presents the architecture verification and status of the implemented platform (Phases 0 to 11 as reported), and an evidence-labelled account of the Phase 12 research, which has been evaluated offline on public datasets but has not been integrated into the hospital-facing platform and therefore remains proposed.

1.4 Objectives of the Project

1. To analyse, specify and document the operational and clinical coordination requirements of a mid-sized hospital as a set of functional and non-functional requirements.

2. To design a multi-tenant, safety-first platform architecture in which every data access is scoped to a single hospital tenant and enforced at the database layer using row-level security.
3. To implement core workflow services for patient administration, appointments with double-booking prevention, caregiver linkage, and role-based access control.
4. To implement clinical-intelligence services for document intake, entity extraction with mandatory clinician verification, allergy and drug-interaction conflict alerting, prioritised care tasks, and tamper-evident prescriptions with adherence tracking.
5. To implement finance and operations services for deterministic insurance coverage estimation and cached, tenant-scoped operational analytics.
6. To implement grounded conversational and optional voice assistance that operates strictly through the platform's own authenticated APIs and never bypasses clinical safeguards.
7. To evaluate, on public multi-channel physiological datasets, a cross-sensor consistency model for detecting coordinated multi-channel false data injection attacks, and to report its results honestly as offline research evidence.
8. To operate the project under a strict phase-gate governance model in which no phase proceeds without an explicit review and authorisation decision.

1.5 Research Questions

The project is guided by the following research questions:

RQ1. How can a hospital operations and clinical coordination platform be structured so that tenant isolation, role-based authorisation and auditability are enforced structurally rather than by convention?

RQ2. How can a conversational assistant be made genuinely useful for hospital navigation while remaining grounded in the hospital's own APIs and incapable of authoring clinical decisions?

RQ3. How can coordinated multi-channel false data injection attacks in IoMT be detected when each individual sensor channel remains within its marginal plausibility bounds?

RQ4. What is the evidence-based status of each phase of the project, and which claims can

be independently verified from the artefacts available in the repository?

RQ1 and RQ2 are addressed by the engineering phases of the project and are answered in Chapters 4 to 6. RQ3 is addressed by the Phase 12 research and is answered in Chapter 8 and Chapter 6. RQ4 is addressed throughout by the evidence-labelling discipline adopted in this report and summarised in Chapter 7 and Chapter 9.

1.6 Scope of the Project

The scope of the implemented and reported platform (Phases 0 to 11) comprises a Next.js 16 web front end, a FastAPI backend, a Supabase PostgreSQL database with row-level security, private object storage for uploaded documents, and an integration with a free-tier LLM provider for grounded tool calling. The scope explicitly includes five user roles: patient, caregiver, doctor, receptionist and hospital administrator. It includes the ten workflow and clinical-intelligence services listed in Section 1.4. It explicitly excludes the following: it does not provide diagnosis, it does not prescribe autonomously, it does not process payments, it does not replace the hospital's statutory medical records system, and it does not implement the CSCM subsystem inside the platform.

The scope of the research component (Phase 12, proposed) comprises the offline design, training and evaluation of a cross-sensor consistency model on two public datasets, and a proposed (not implemented) read-only inference adapter and advisory alerting path. The research component does not involve live patient data and has not been deployed against any hospital device.

1.7 Contributions

The principal contributions of the project are summarised below. Each contribution is labelled with its verification status in accordance with the evidence hierarchy used in this report (source code, test logs, migrations and configuration, phase reports, then plans).

Table 1.1 Contribution, phase and verification status

Contribution	Phase	Verification status
A multi-tenant schema with structural row-level security and composite foreign keys, verified from migration SQL.	Phase 2	Implemented, verifiable from migration source.
Workflow services for patients, appointments with double-booking	Phase 3	Implemented, verifiable from router source.

Contribution	Phase	Verification status
prevention, caregiver linkage and organisation management.		
A five-role front end covering patient portal, caregiver view, doctor workstation, receptionist desk and admin console.	Phase 4	Implemented, verifiable from front-end source.
A grounded conversational agent with server-side tool calling, emergency escalation and an offline circuit breaker.	Phase 5	Implemented, verifiable from agent source and phase test log.
Document intake with extraction and mandatory clinician verification before records are released.	Phase 6	Implemented, verifiable from router and storage source.
Deterministic, advisory allergy and drug-drug-interaction conflict detection with acknowledgement workflow.	Phase 7	Implemented, verifiable from knowledge-base and detector source.
Prioritised care tasks with claim, complete and cancel lifecycle and notification counts.	Phase 8	Implemented, verifiable from router source.
Tamper-evident prescriptions with HMAC signing, public verification and adherence tracking.	Phase 9	Implemented, verifiable from crypto and adherence source.
Conversational assistant plus optional, feature-flagged voice prototype on the same safety rails.	Phase 10	Implemented, verifiable from chat router and phase report.
Deterministic insurance estimation with explainable breakdown, and cached operations analytics.	Phase 11	Implemented, verifiable from rules and cache source.
A physiology-aware cross-sensor consistency model for coordinated FDIA detection evaluated on PhysioNet 2012 and WESAD.	Phase 12 (proposed)	Offline research evidence only; not integrated; not independently verified in this report.

1.8 Organisation of the Report

Chapter 2 reviews the literature relevant to each subsystem and to IoMT attack detection, and identifies the research gap. Chapter 3 presents the requirement analysis, including functional and non-functional requirements and the requirements traceability matrix. Chapter 4 presents the system design, including the architecture, entity-relationship model, API catalogue, security and threat model. Chapter 5 presents the implementation of Phases 0 to 14, with the evidence-based status of each phase. Chapter 6 describes the algorithms and pseudocode, including the CSCM method. Chapter 7 describes the testing and evaluation strategy and the evidence collected. Chapter 8 presents the results and discussion, including the offline CSCM evaluation. Chapter 9 presents project management, schedule, milestones, cost analysis and risk management. Chapter 10 concludes and outlines future work. The report ends with references in APA style and appendices containing sample code, configuration and the submission-readiness checklist.

CHAPTER 2
LITERATURE REVIEW

2.1 Introduction

This chapter reviews the literature underpinning each subsystem of CareBridge AI and the research component. The review is organised by domain, moving from platform-level concerns (hospital information systems, multi-tenancy, conversational assistance) through clinical intelligence (document understanding, decision support, prescription integrity) to the research core (IoMT security, false data injection, and time-series and graph-based anomaly detection). For each domain the review states what is established, and then identifies what remains open. The chapter closes with the explicit research gap that CareBridge AI addresses.

Two conventions are used throughout. First, engineering claims that originate from the project's own phase reports are described as reported, because this report's evidence hierarchy places source code and test logs above narrative reports. Second, each external work is cited in APA style and listed in the References. Where a work is used only to establish general background, that is stated explicitly so that the reader can separate context from contribution.

2.2 Hospital Information Systems, EHRs and Multi-Tenancy

The digitisation of hospital operations has been studied extensively. Rajkomar et al. (2018) demonstrated that deep learning applied to the longitudinal sequence of events in an EHR can predict outcomes such as in-hospital mortality and readmission, establishing that EHR data has predictive structure worth modelling. Topol (2019) surveyed the convergence of artificial intelligence and medicine and argued that the highest near-term value lies in augmenting clinicians rather than replacing them, a position that directly informs the human-in-the-loop stance of CareBridge AI. More recently, the study of clinical decision support systems by Sutton et al. (2020) catalogued both the benefits and the failure modes of such systems, emphasising that alert fatigue and poor workflow integration are the dominant causes of under-utilisation. CareBridge AI responds to that finding by making its conflict alerts advisory, evidence-bearing and explicitly acknowledgeable.

From a software architecture standpoint, multi-tenancy is the dominant pattern for Software-as-a-Service healthcare platforms. Taibi, Lenarduzzi and Pahl (2018) catalogued

architectural patterns for microservices and their trade-offs, and observed that data isolation is the most consequential decision in a multi-tenant system. Two isolation strategies dominate: a shared schema with a tenant discriminator column, and a schema-per-tenant design. The shared-schema approach with a discriminator column is the more common because it keeps migrations cheap, but it places the burden of correctness on every query. Row-level security (RLS), a declarative feature of PostgreSQL, moves that burden from application code to the database engine by attaching a policy to each table that is evaluated automatically for every statement. This is the strategy adopted in CareBridge AI, and it is discussed in detail in Chapter 4.

A recurring concern in the literature is interoperability. HL7 Fast Healthcare Interoperability Resources (FHIR) (HL7 International, 2023) is the dominant standard for exchanging clinical data. CareBridge AI does not currently expose a FHIR interface; this is recorded as a limitation in Chapter 10 and as a deferred item in the phase plan, because implementing a conformant FHIR façade is a substantial undertaking that was outside the eighteen-week project window.

2.3 Conversational AI and Large Language Model Agents in Healthcare

Large language models have moved rapidly from general text generation to domain-competent assistants. Singhal et al. (2023) evaluated a medical question-answering model and reported strong performance on clinical knowledge benchmarks while cautioning that automated benchmarks alone are insufficient and that human evaluation is indispensable. Thirunavukarasu et al. (2023) reviewed large language models in medicine and identified hallucination, brittleness and the absence of grounding as the principal barriers to clinical deployment. Their conclusion is directly relevant: an assistant that is allowed to answer from parametric memory alone is unsafe in a hospital context.

Two techniques address the grounding problem. Retrieval-Augmented Generation (Lewis et al., 2020) conditions generation on documents retrieved from an external corpus, and ReAct (Yao et al., 2023) interleaves reasoning with tool actions so that the model can query authoritative systems rather than recall facts. Toolformer (Schick et al., 2023) showed that a model can learn to call external tools. CareBridge AI adopts the tool-calling position in its strongest form: the conversational assistant is not given the ability to answer operational

questions from memory. Instead, every operational fact is retrieved through the platform's own authenticated APIs, executed server-side under the caller's security context, so that tenant isolation and role permissions are enforced by the same code path that serves the user interface. This design choice, described in Chapter 6, is the practical realisation of the grounding requirement identified in the literature.

Table 2.1 Representative large language model and conversational studies and their relevance

Study	Focus	Relevance to CareBridge AI
Singhal et al. (2023)	Clinical knowledge of LLMs; human evaluation	Supports human-in-the-loop review and caution over automated benchmarks.
Thirunavukarasu et al. (2023)	Review of LLMs in medicine	Motivates grounding and hallucination controls.
Lewis et al. (2020)	Retrieval-augmented generation	Motivates conditioning answers on authoritative sources.
Yao et al. (2023)	Reason and act (tool use)	Directly motivates server-side tool calling.
Schick et al. (2023)	Tool learning	Supports the plausibility of tool-oriented assistants.

2.4 Document Intelligence in Clinical Settings

Clinical documents arrive as scans, photographs and portable document format (PDF) files. Optical character recognition (OCR) is the established technology for converting raster text to machine-readable text; the Tesseract engine described by Smith (2007) remains a widely used open-source baseline. Modern document pipelines combine OCR with rule-based or statistical entity extraction to recover structured facts such as medication names, dosages and dates. In a clinical setting, automated extraction is not sufficient on its own: an incorrectly extracted allergy or dosage can be more dangerous than no extraction at all. The literature on clinical decision support (Sutton et al., 2020; Kuperman et al., 2007) is consistent in recommending that machine-extracted clinical facts be reviewed by a responsible human before they influence care. CareBridge AI implements exactly this separation: extracted entities are stored in a provisional state and are only promoted to authoritative records after an explicit clinician verification step, as described in Chapters 4 and 5.

2.5 Clinical Decision Support: Allergy and Drug-Interaction Alerting

Medication-related harm is a leading cause of preventable adverse events, and computerised provider order entry with decision support reduces it, although not as reliably as early optimism suggested (Kuperman et al., 2007). Payne et al. (2015) analysed why drug-drug interaction

alerts are frequently overridden and produced concrete recommendations: alerts should be tiered by severity, should show the reason, and should be presented at the moment of decision. Sittig and Singh (2010), in their sociotechnical model, argued that health information technology failures are usually the product of mismatches between technology and the surrounding social system rather than pure technical faults. Together these works define the design constraints for CareBridge AI's conflict subsystem: alerts are advisory, severity-tiered, evidence-bearing, and always acknowledgeable with a rationale. The system deliberately does not attempt to be an exhaustive pharmacological reference; it implements a curated, auditable knowledge base of high-consequence rules, which is described in Chapter 6.

2.6 Prescription Integrity and Medication Adherence

Paper and low-integrity digital prescriptions are vulnerable to undetectable alteration and to forgery. Cryptographic message authentication, in particular keyed hashing with HMAC, provides a standard mechanism for detecting tampering: a verifier who knows the key can recompute the digest and confirm that the content is unchanged. CareBridge AI applies this idea in a deliberately narrow and testable way. Each prescription item set is serialised canonically and signed, and a public verification endpoint allows a pharmacist to confirm authenticity without exposing any patient-identifying information, as described in Chapters 4 and 5. Adherence, defined as the extent to which a patient's medication-taking behaviour corresponds to the prescribed regimen, is a well-studied but difficult problem; self-report is unreliable, and the literature generally favours a combination of simple objective proxies and reminder-based intervention over elaborate inference. CareBridge AI therefore records adherence events and computes simple, transparent adherence summaries rather than claiming to measure true ingestion.

2.7 Insurance Coordination and Financial Estimation

The administrative cost of healthcare is a recognised global problem, and the literature on healthcare analytics (Raghupathi and Raghupathi, 2014) frames financial coordination as a data-integration challenge as much as a financial one. In the Indian context, insurance and reimbursement workflows are highly heterogeneous, and any automated estimate must be transparent about its assumptions. CareBridge AI therefore implements a deterministic, versioned rule engine that produces an explainable breakdown and explicitly distinguishes

three outcomes: insufficient information, review required, and estimate available. This conservative posture is consistent with the literature's finding that opaque financial automation erodes trust.

2.8 Healthcare Operations Analytics

Raghupathi and Raghupathi (2014) argued that analytics in healthcare creates value primarily by improving operational decision-making, and that the principal barrier is data quality rather than algorithmic sophistication. Operationally, dashboards that answer simple, well-defined questions quickly are more valuable than sophisticated models that answer ambiguous questions slowly. CareBridge AI follows this guidance: its analytics layer computes a small number of well-defined hospital-level aggregates, exposes them through a narrow API, and caches them in a tenant-scoped bounded time-window cache to keep the surface responsive, as described in Chapters 4 and 5.

2.9 IoMT Security and False Data Injection Attacks

The Internet of Medical Things extends networked computation to physiological sensors, infusion pumps, wearables and implantable devices. This connectivity enlarges the attack surface dramatically. Alsubaei, Abuhussein and Shiva (2017) developed a taxonomy and risk assessment for IoMT security and showed that device constraints make conventional host-based defences impractical. A specific and serious class of attack is the false data injection attack (FDIA), in which the adversary corrupts the measurements that a monitoring or estimation process consumes, so that the process produces incorrect outputs while appearing to operate normally. FDIA was formalised for power-grid state estimation by Liu, Ning and Reiter (2011), who showed that an adversary with knowledge of the system model can construct injections that evade conventional bad-data detection. The analogy to physiological monitoring is direct: a monitor that trusts a single channel can be deceived by an injection that keeps that channel plausible. Mo et al. (2012) extended this line of analysis to the broader cyber-physical smart-grid infrastructure and demonstrated the importance of system-model knowledge for both constructing and detecting such attacks.

The distinguishing feature of the study in this report is coordination across channels. In a multi-sensor physiological setup, sensors are not independent: heart rate, pulse wave morphology, respiratory rate and temperature are physiologically coupled. An adversary who

corrupts several channels simultaneously, for example by scaling a subset of them by modest factors, can keep each channel inside its marginal bounds while producing a joint state that is physiologically implausible. Single-channel outlier detectors and univariate plausibility rules cannot see this, because each channel, considered alone, looks normal. Detecting such an attack requires a model of the joint distribution, or equivalently of the consistency relations between channels.

2.10 Time-Series and Graph-Based Anomaly Detection

Deep anomaly detection is a mature field. Chalapathy and Chawla (2019) surveyed the area and grouped methods into reconstruction-based, density-based and one-class classification families, noting that the choice of model must match the structure of the data. Physiological signals are temporal, so recurrent and convolutional sequence models are natural encoders; the Gated Recurrent Unit (GRU) introduced by Cho et al. (2014) is a widely used, computationally economical recurrent unit, and the transformer architecture of Vaswani et al. (2017) provides an attention-based alternative. Sensor relationships, on the other hand, are relational, so graph-based models are natural. Kipf and Welling (2017) introduced the graph convolutional network, Gilmer et al. (2017) generalised message passing, and Battaglia et al. (2018) argued that graph networks provide a powerful relational inductive bias. The CSCM method described in Chapter 6 combines a temporal encoder with an inter-sensor consistency graph, which places it at the intersection of the temporal and relational families.

Table 2.2 Families of anomaly detection and their applicability to IoMT FDIA

Family	Representative ideas	Limitation for coordinated FDIA
Statistical / threshold	Per-channel bounds, robust z-scores	Blind to coordinated multi-channel attacks inside bounds.
Reconstruction	Autoencoders, PCA residual error	A coordinated attack can reconstruct consistently if the model has no cross-channel constraint.
Density / one-class	Kernel and deep one-class methods	High-dimensional joint densities require large data and are hard to calibrate.
Temporal sequence	RNN, GRU, transformer	Model temporal plausibility but not cross-channel physiology explicitly.
Graph / relational (CSCM)	Message passing over a sensor graph	Directly models cross-channel consistency; the focus of Phase 12.

2.11 Research Gap

The review supports the following synthesis. On the engineering side, the building blocks of a safe, multi-tenant hospital platform are individually well understood, but their assembly under

a strict, structural isolation and human-in-the-loop discipline remains a non-trivial systems problem, and published accounts of complete, low-cost implementations are scarce. On the research side, FDIA detection is well developed for physical infrastructure such as power grids, and deep anomaly detection is well developed for single or loosely coupled time series, but the specific problem of detecting coordinated, within-bounds manipulation of multiple coupled physiological channels remains comparatively underexplored. Existing single-channel and reconstruction methods are structurally ill-suited to it, because the attack is defined by a joint inconsistency that marginal models do not represent.

CareBridge AI's research contribution is therefore to model, learn and evaluate cross-sensor consistency for coordinated multi-channel FDIA detection in IoMT, using two independent public datasets to test generalisation. The engineering contribution is a complete, evidence-documented, safety-first platform that provides the operational context in which such a detector would eventually be deployed, while keeping the detector itself clearly separated and clearly labelled as proposed. Neither contribution is overstated: the platform phases are reported as complete but are labelled as reported rather than independently verified, and the detector is labelled as evaluated offline and not integrated.

2.12 Summary

This chapter established the state of the art across the platform and research domains, and identified the gap that CareBridge AI addresses. Chapter 3 converts these findings into a concrete, testable requirement specification.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 Introduction

This chapter defines what CareBridge AI must do and the qualities it must exhibit. Requirements were elicited from the Phase 0 audit of the project, from the operational weaknesses enumerated in Chapter 1, and from the literature reviewed in Chapter 2. Requirements are numbered so that every subsequent design decision, implementation module and test can be traced back to a requirement, and so that the requirements traceability matrix at the end of the chapter can demonstrate coverage.

3.2 Requirement Elicitation Approach

Because the project has no access to live hospital data, elicitation was necessarily indirect. The Phase 0 audit produced a baseline inventory of the repository and of the problem domain. The problem areas of Section 1.2 were then decomposed into concrete capabilities, and each capability was interrogated with three questions: what must the system do, what must it never do, and how will we know that it does it. This produced the three requirement classes used below: functional requirements, non-functional requirements and prohibitions (safety constraints expressed as requirements that forbid behaviour). Where a requirement could not be verified from an available artefact, it is marked accordingly in the traceability matrix.

3.3 Stakeholders and Roles

The platform recognises five primary roles, each with a distinct information need and a distinct permission envelope. The role is a property of the authenticated profile and is enforced both at the API layer and, more fundamentally, at the database layer.

Table 3.1 Stakeholder roles and their primary concerns

Role	Primary concern	Representative capabilities
Patient	Understand and manage their own care	View own appointments, documents, prescriptions, adherence and insurance estimate.
Caregiver	Support a linked patient	Request a link to a patient, verify it, view permitted information, and have it revocable at any time.
Doctor	Clinical safety and efficiency	Review the exam queue and vitals, verify extracted document entities, author prescriptions, acknowledge conflicts.

Role	Primary concern	Representative capabilities
Receptionist	Smooth front-desk operations	Register patients, schedule and reschedule appointments, manage documents intake.
Hospital Administrator	Operational oversight	View capacity, doctors, departments and cached analytics; manage organisation data.

3.4 Functional Requirements

Functional requirements describe observable behaviour. Each is assigned an identifier of the form FR-nn and is traceable to at least one design element and one verification artefact.

Table 3.2 Functional requirements

ID	Requirement	Phase
FR-01	The system shall authenticate users and derive a tenant-scoped security context containing user identity, hospital identity and role.	2
FR-02	The system shall enforce row-level isolation so that no request can read or write another hospital's data.	2
FR-03	The system shall maintain hospital, department, doctor, patient and profile records with unique keys per tenant.	2, 3
FR-04	The system shall register patients with a tenant-unique medical record number and demographic details.	3
FR-05	The system shall allow a patient or receptionist to search doctors and view available slots.	3
FR-06	The system shall prevent double-booking of a doctor for overlapping appointment intervals.	3
FR-07	The system shall enforce a cancellation cut-off policy for appointments.	3
FR-08	The system shall allow a caregiver to request a link to a patient, which a responsible party can verify or revoke.	3
FR-09	The system shall accept document uploads with content-type validation and a size limit, storing them in tenant-isolated private storage.	6
FR-10	The system shall extract text and candidate entities from uploaded documents and store them as provisional.	6
FR-11	The system shall require an authorised clinician to verify, edit or reject extracted entities before they become authoritative.	6
FR-12	The system shall scan a patient's verified record for allergy and drug-drug-interaction conflicts and record each finding with its evidence.	7

ID	Requirement	Phase
FR-13	The system shall allow a clinician to acknowledge a conflict with a rationale and shall retain an audit trail.	7
FR-14	The system shall create, assign, claim, complete and cancel care tasks with priority and notification counts.	8
FR-15	The system shall author prescriptions as item sets and produce a tamper-evident verification digest.	9
FR-16	The system shall expose a public verification endpoint that confirms prescription authenticity without revealing patient-identifying information.	9
FR-17	The system shall record adherence events and compute a transparent adherence summary.	9
FR-18	The system shall compute a deterministic insurance coverage estimate with an explicit breakdown, or return insufficient information, or flag review required.	11
FR-19	The system shall compute tenant-scoped operational analytics aggregates and serve them through a cached endpoint.	11
FR-20	The system shall provide an authenticated conversational assistant that answers operational questions only by calling the platform's own APIs.	5, 10
FR-21	The system shall detect emergency-indicating input and escalate with emergency contact guidance.	5
FR-22	The system shall degrade gracefully when the language model provider is unavailable, via a circuit breaker and deterministic fallback.	5
FR-23	The system shall expose an optional voice interface that remains disabled unless explicitly configured.	10

3.5 Non-Functional Requirements

Non-functional requirements describe qualities and constraints. They are the requirements most directly responsible for the platform's safety posture, and several of them are expressed as prohibitions.

Table 3.3 Non-functional requirements

ID	Category	Requirement
NFR-01	Isolation	Tenant isolation shall be enforced at the database layer by row-level security and shall not depend on application-layer filtering alone.

ID	Category	Requirement
NFR-02	Authorization	Every mutating endpoint shall enforce a role check; unauthorised roles shall be rejected before any data access.
NFR-03	Auditability	Security-relevant and clinical-coordination actions shall emit structured audit records.
NFR-04	Safety (prohibition)	The system shall not author a clinical decision; all conflict, insurance and attack-detection outputs shall be advisory.
NFR-05	Safety (prohibition)	The assistant shall not answer operational questions from model memory; it shall use tools.
NFR-06	Privacy	Personally identifiable information shall not be exposed by the public prescription-verification endpoint.
NFR-07	Error handling	Errors shall be returned as a consistent, machine-readable problem document without leaking internal state.
NFR-08	Performance	Cached analytics reads shall typically complete in well under one second; cache hits shall be served without a database query.
NFR-09	Cost	The platform shall operate at zero marginal cost by using free tiers for the language model and managed database.
NFR-10	Maintainability	The backend shall be organised as cohesive modules with a thin routing layer and testable service functions.
NFR-11	Portability	The platform shall run on a Windows or Linux developer workstation with documented setup steps.
NFR-12	Usability	The front end shall present role-appropriate navigation and shall not display controls the role cannot use.

3.6 Use Cases

The most consequential use cases are summarised below. Each names the primary actor, the trigger, the main success path, and the safety-relevant alternative path. The detailed flows appear in Chapter 4 as sequence diagrams.

Table 3.4 Principal use cases

Use case	Actor	Summary
UC-1 Book appointment	Receptionist, Patient	Search doctors, choose a slot, confirm. Alternative: the slot overlaps an existing booking and the database rejects the insert.
UC-2 Verify document	Doctor	Review provisional entities, approve or correct them, and thereby release the record to the patient timeline.
UC-3 Acknowledge conflict	Doctor	View a tiered conflict alert with evidence and acknowledge it with a rationale.

Use case	Actor	Summary
UC-4 Sign and verify prescription	Doctor, Pharmacist	Author items, produce a digest, and verify authenticity publicly without exposing PII.
UC-5 Estimate coverage	Receptionist, Administrator	Enter amount and category, receive a deterministic breakdown or an explicit insufficiency/review outcome.
UC-6 Ask the assistant	Any authenticated role	Ask an operational question; the assistant answers only via tools executed under the caller's security context.
UC-7 Detect coordinated FDIA	Biomedical engineering (proposed)	Obtain an advisory indication that cross-sensor consistency is violated; the absence of an alert is explicitly not evidence of safety.

3.7 Technology Stack

The technology choices are driven by the zero-marginal-cost constraint and by the need for a database that can enforce tenant isolation declaratively.

Table 3.5 Technology stack

Layer	Technology	Rationale
Front end	Next.js 16 with React 19 and TypeScript	Server components, routing, and static compilation; typed API clients.
Backend	FastAPI on Python	Asynchronous routing, dependency injection for security context, automatic OpenAPI.
Database	Supabase PostgreSQL	Managed PostgreSQL with authentication and row-level security.
Isolation	PostgreSQL row-level security	Declarative, engine-enforced tenant isolation.
Storage	Private object storage bucket	Server-mediated access to uploaded documents.
Language model	Free-tier hosted LLM with tool calling	Grounded assistant without marginal cost.
Document processing	PDF text extraction with OCR fallback	Recover text from born-digital and scanned documents.
Research	PyTorch	Model training and evaluation for the CSCM experiment.
Version control	Git	Traceable history of the repository.

3.8 Constraints and Assumptions

The project operates under explicit constraints, and the assumptions are stated so that they can be challenged. The constraints are: an eighteen-week delivery window; a zero-marginal-cost budget; no access to live patient data; and a single developer. The assumptions are: that the hospital uses a modern browser; that reception staff can operate a web form; that clinicians will review extracted entities; that prescriptions are authored digitally before being printed; and that any future IoMT deployment will provide timestamped multi-channel windows with

reasonably stable sampling.

3.9 Requirements Traceability Matrix

The traceability matrix links each requirement to the design element that realises it, the implementation module in which it lives, and the artefact from which it can be verified. The verification column names the strongest available evidence, in accordance with the evidence hierarchy of this report. Requirements whose verification rests on a phase report rather than source code or test output are marked as reported.

Table 3.6 Requirements traceability matrix

Requirement	Design element	Implementation module	Verification evidence
FR-01, FR-02	Security context and RLS (Ch. 4)	core/security, migrations 0001	Migration SQL and security source (source code).
FR-03, FR-04	Tenant schema (Ch. 4)	patients, organization routers	Router source and migration SQL.
FR-05, FR-06, FR-07	Appointment model (Ch. 4)	appointments router	Router source; exclusion constraint in migration SQL.
FR-08	Caregiver link (Ch. 4)	caregiver router	Router source.
FR-09, FR-10, FR-11	Document pipeline (Ch. 4)	documents router, storage	Router source.
FR-12, FR-13	Conflict engine (Ch. 4)	conflicts router, knowledge base	Knowledge-base source.
FR-14	Care task lifecycle (Ch. 4)	care_tasks router	Router source.
FR-15, FR-16, FR-17	Prescription integrity (Ch. 4)	prescriptions router, crypto	Crypto source and migration SQL.
FR-18	Insurance rule engine (Ch. 4)	insurance rules	Rule source with version tag.
FR-19	Analytics and cache (Ch. 4)	analytics service, cache	Cache source with TTL and eviction.
FR-20, FR-22	Agent engine (Ch. 4)	agent engine, client	Agent source and phase test log (reported).
FR-21	Emergency escalation (Ch. 4)	agent engine	Agent source and phase test log (reported).
FR-23	Voice toggle (Ch. 4)	chat router	Chat router source; disabled by default.
NFR-01, NFR-02	RLS and role barriers	migrations, core/security	Migration SQL; role guards in routers.
NFR-03	Audit service	core/audit	Audit source.
NFR-04, NFR-05	Advisory posture	agent prompts, conflict detector	Prompt and detector source.
NFR-06	Public verification	prescriptions crypto	Constant-time compare over hash only.
NFR-08	Bounded cache	analytics cache	Cache source (TTL 60 s, 256 entries).
NFR-09	Zero cost	config, provider choice	Configuration and phase reports (reported).

3.10 Requirement Prioritisation

Not every requirement carries the same delivery risk. The requirements were therefore prioritised so that the safety-critical core would be delivered first and the discretionary

capability could be deferred without weakening the isolation or clinical-safety guarantees. Table 3.7 applies a must, should and could classification. The must set is the set of requirements without which the platform could not be operated safely; it is also the set that the phase gates treated as non-negotiable. The should set improves clinical coordination and operational oversight but can be delivered incrementally. The could set is genuinely optional and is the first place that scope would be reduced if the schedule were threatened.

Table 3.7 Requirement prioritisation

Priority	Requirements	Rationale
Must	FR-01, FR-02, FR-03, FR-04, FR-05, FR-06, FR-07, FR-15, FR-16, NFR-01 to NFR-06	Isolation, authorisation, identity, safe scheduling, prescription integrity and the safety prohibitions. Failure here is a correctness or privacy failure, not a missing feature.
Should	FR-09 to FR-14, FR-17, FR-18, FR-20, FR-21, FR-22, NFR-07, NFR-10	Clinical coordination capabilities that materially improve care but whose absence would not corrupt the record or leak data.
Could	FR-08, FR-19, FR-23, NFR-08, NFR-11, NFR-12	Convenience and oversight features, optional voice, analytics caching and portability that can be layered on once the must and should sets are stable.

The prioritisation explains an aspect of the delivery sequence that might otherwise look surprising. Prescription integrity, which is a must, was delivered in Phase 9, while caregiver linkage, which is a could, was delivered in Phase 3. The ordering reflects dependency and risk rather than priority alone: caregiver linkage is cheap to build once the patient schema exists, whereas prescription integrity required the cryptographic discipline and the verification workflow that were only ready later. Priority governed what had to be correct; dependency governed what could be built when.

3.11 Summary

This chapter established a numbered, traceable requirement set covering 23 functional requirements, 12 non-functional requirements and 7 principal use cases, each linked to design elements, implementation modules and available verification evidence. Chapter 4 presents the design that satisfies these requirements.

CHAPTER 4

SYSTEM DESIGN

4.1 Introduction

This chapter presents the design of CareBridge AI. It begins with the design principles that govern every decision, then describes the high-level architecture, the deployment topology, the relational data model, the application programming interface (API), the authentication and authorisation flow, six detailed interaction designs, the security and threat model, and finally the design of the proposed Phase 12 integration. The design is presented as a set of diagrams and specification tables so that each element can be traced to the requirements of Chapter 3 and to the implementation described in Chapter 5.

4.2 Design Principles

Six principles shape the design. They are stated here because they explain many otherwise surprising choices.

1. Structural isolation over conventional isolation. Tenant isolation is enforced by PostgreSQL row-level security (PostgreSQL Global Development Group, 2024) rather than by application-layer filters, so that a missing filter in a new endpoint cannot leak data across hospitals.
2. Mediated access. The front end never talks to the database directly for privileged operations. It calls the backend, which derives a security context and executes the query in the caller's identity, allowing row-level security to apply.
3. Human-in-the-loop for order-sensitive decisions. Extraction, conflict detection, insurance estimation and attack detection all produce advisory output that a human must act upon. The system does not silently persist a clinical conclusion.
4. Grounded assistance. The conversational assistant can only report facts that a tool retrieved from the platform's own APIs under the caller's security context.
5. Fail safe and fail cheap. Provider outages degrade to deterministic fallbacks rather than to errors; the entire platform is designed to run within free tiers.
6. Evidence before claims. No capability is described as working unless a source artefact,

a test log or a migration supports it.

4.3 High-Level Architecture

Figure 4.1 places CareBridge AI in its environment. Five human roles interact with the platform. The platform integrates with a hosted language model, a managed PostgreSQL database with authentication, private object storage, the proposed CSCM model service, and, for emergency escalation only, the public emergency numbers. The proposed CSCM service is drawn with a dashed relationship and labelled as proposed to avoid any implication that it is operational.

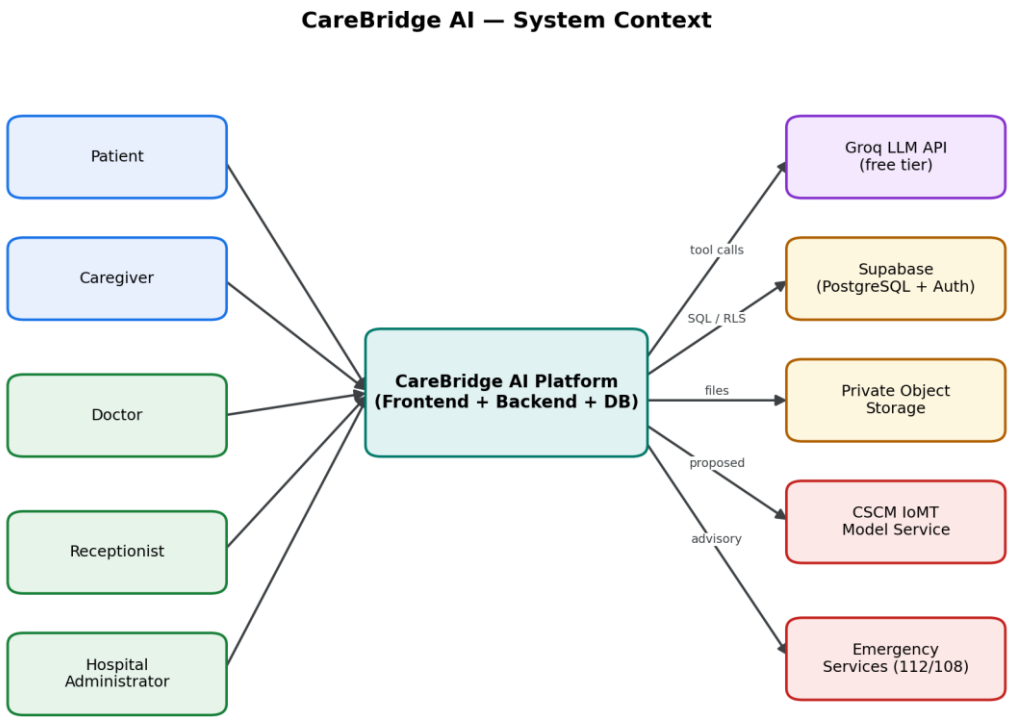


Fig. 4.1 System context of CareBridge AI showing actors, the platform and external services

Figure 4.2 decomposes the platform into components. The Next.js front end renders five role-specific surfaces. The FastAPI backend is organised into a routing layer, a set of domain modules, a shared core, and a research-facing CSCM adapter. The domain modules are grouped into workflow services, clinical-intelligence services, and finance and operations services. The core provides the security context, database access and audit service that all modules share.

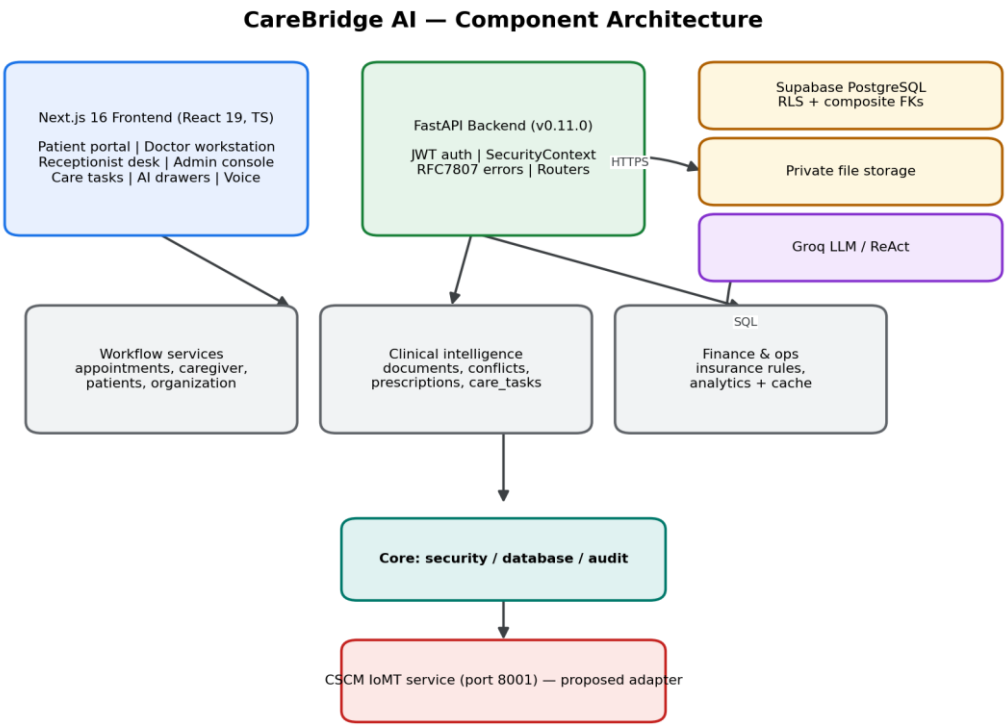


Fig. 4.2 Component architecture of CareBridge AI

The separation between routing and domain modules is deliberate. Routers are kept thin: they validate input, obtain the security context, call a domain function, and translate domain errors into problem responses. Domain functions contain the logic and are written so that they can be reasoned about and tested without an HTTP server. This mirrors the maintainability requirement NFR-10 and is the structure that made the phase tests of Chapter 7 possible.

4.4 Deployment View

Figure 4.3 shows the deployment topology. In development, all components run on a single workstation: the Next.js development server on port 3000, the FastAPI backend on port 8000, and the proposed CSCM service on port 8001. In a hosted configuration, the front end and backend are deployed independently, while the database, storage and language-model provider remain managed services. The design deliberately avoids any requirement for a long-running background worker, which keeps the operational cost at zero.

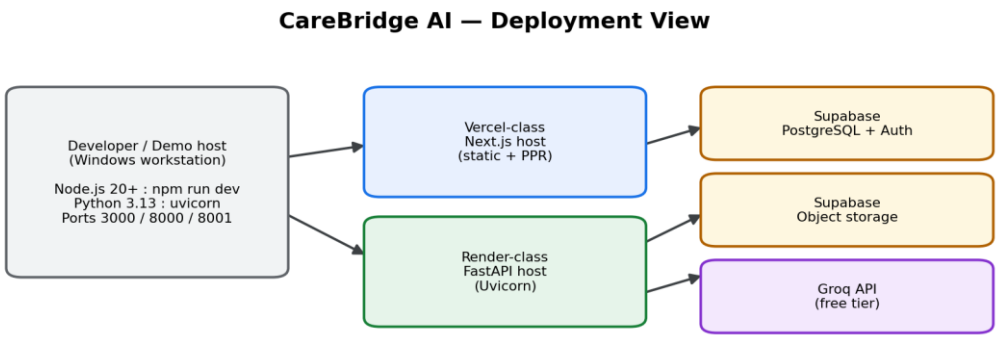


Fig. 4.3 Deployment view of the development and hosted configurations

4.5 Data Model

The data model is the backbone of the isolation guarantee. Every tenant-scoped table carries a hospital_id column, and every relationship between tenant-scoped tables is expressed as a composite foreign key on (hospital_id, id). Because the referenced table exposes a unique constraint on (hospital_id, id), the database itself guarantees that a row can never reference a parent belonging to another hospital. This is a stronger guarantee than an application check, because it cannot be bypassed by a coding mistake.

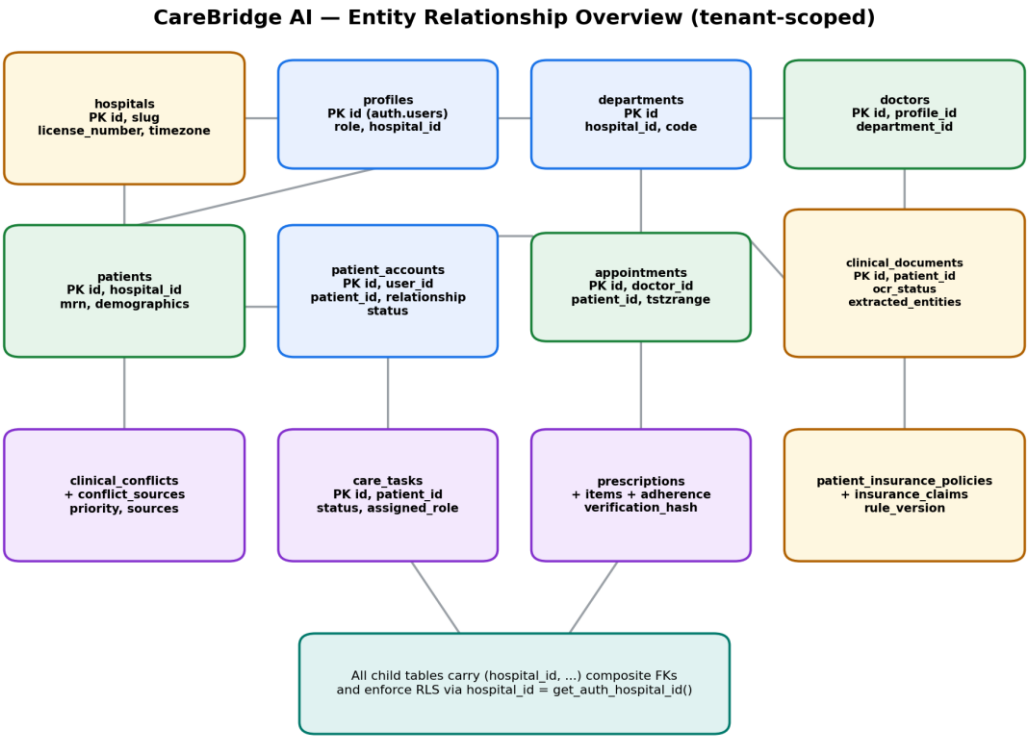


Fig. 4.4 Entity-relationship overview of the tenant-scoped schema

Table 4.1 lists the seventeen tables that constitute the schema, together with their purpose and their strongest isolation mechanism. The schema was created by seven forward migrations, each paired with a down migration, which is a practice that keeps every phase reversible.

Table 4.1 Database tables and their isolation mechanisms

Table	Purpose	Isolation mechanism
hospitals	Tenant root record	RLS policy: id = get_auth_hospital_id()
departments	Departments within a hospital	RLS on hospital_id; composite unique (hospital_id, code)
profiles	Authenticated user profiles and roles	RLS; trigger protecting privilege fields
doctors	Clinician records linked to profile and department	Composite FKs to profiles and departments
patients	Patient demographics and MRN	RLS with has_patient_access(); composite unique (hospital_id, mrn)
patient_accounts	Caregiver and self links to patients	RLS; composite FKs; unique (user_id, patient_id)
appointments	Scheduled encounters	RLS; GiST exclusion constraint preventing double-booking
clinical_documents	Uploaded documents and OCR state	RLS; composite FK to patients
extracted_entities	Provisional extracted facts	RLS; verification_status workflow
clinical_conflicts	Detected advisory conflicts	RLS; composite FK to patients
clinical_conflict_sources	Evidence supporting each conflict	RLS; composite FKs to document and entity

Table	Purpose	Isolation mechanism
care_tasks	Prioritised operational tasks	RLS; assigned_role and status lifecycle
prescriptions	Prescription headers with verification hash	RLS; composite FK to patients
prescription_items	Individual medication items	RLS; composite FK to prescriptions
medication_adherence_logs	Adherence events per item	RLS; composite FKs to patient and item
patient_insurance_policies	Insurance policy parameters	RLS; composite FK to patients
insurance_claims	Claims and calculated breakdowns	RLS; rule_version and JSONB breakdown

Three details deserve emphasis. First, the exclusion constraint on appointments (exclude_doctor_double_booking) uses a GiST index over the doctor identifier and a half-open timestamp range, restricted by a predicate that ignores cancelled and no-show appointments; this makes double-booking impossible at the storage layer rather than merely unlikely. Second, extracted entities carry a verification_status with the values pending, verified, rejected and edited, which is the mechanism that keeps machine-extracted facts provisional until a clinician acts. Third, prescriptions carry a verification_hash in the form version-prefixed hexadecimal HMAC, which is the tamper-evidence mechanism described in Section 4.8.5.

4.6 API Design

The backend exposes a versioned REST API under the path prefix /api/v1. Every request is authenticated by a bearer token; every response is either the requested representation or a machine-readable problem document. Table 4.2 catalogues the endpoints by module. The catalogue was extracted directly from the router source, so it reflects the implementation rather than the plan.

Table 4.2 API endpoint catalogue

Module	Method and path	Purpose
organization	GET /hospitals/me	Return the caller's hospital profile.
organization	GET /departments	List departments in the caller's hospital.
doctors	GET /doctors	Search doctors by name, department or specialisation.
doctors	GET /doctors/{doctor_id}/slots	List available appointment slots for a doctor.
patients	POST /patients	Register a patient with a tenant-unique MRN.
patients	GET /patients	Search patients within the tenant.
patients	GET /patients/{patient_id}	Retrieve a patient record.
appointments	POST /appointments	Create an appointment, subject to the exclusion constraint.
appointments	GET /appointments	List appointments visible to the caller.
appointments	PATCH /appointments/{id}/status	Transition appointment status, enforcing policy.

Module	Method and path	Purpose
appointments	POST /appointments/{id}/reschedule	Move an appointment to a new slot.
patient-accounts	POST /patient-accounts/request	Request a caregiver link to a patient.
patient-accounts	POST /patient-accounts/{id}/verify	Verify a pending link.
patient-accounts	PATCH /patient-accounts/{id}/revoke	Revoke an existing link.
documents	POST /documents/upload	Upload a document with type and size validation.
documents	GET /documents	List documents visible to the caller.
documents	GET /documents/{id}	Retrieve document metadata.
documents	GET /documents/{id}/entities	List provisional extracted entities.
documents	POST /documents/{id}/verify	Verify, edit or reject an entity (clinician only).
documents	GET /documents/{id}/download	Download the stored file through the server.
documents	GET /documents/notifications/unread-count	Unread document notifications.
conflicts	POST /conflicts/scan/{patient_id}	Run the advisory conflict scan and persist findings.
conflicts	GET /conflicts	List conflicts visible to the caller.
conflicts	GET /conflicts/{id}	Retrieve a conflict with its evidence sources.
conflicts	POST /conflicts/{id}/acknowledge	Acknowledge a conflict with a rationale.
conflicts	GET /conflicts/notifications/counts	Conflict notification counts.
care-tasks	POST /care-tasks	Create a prioritised care task.
care-tasks	GET /care-tasks	List care tasks filtered by role and status.
care-tasks	GET /care-tasks/{id}	Retrieve a care task.
care-tasks	POST /care-tasks/{id}/claim	Claim a task for the caller.
care-tasks	POST /care-tasks/{id}/complete	Complete a task with notes.
care-tasks	POST /care-tasks/{id}/cancel	Cancel a task with a reason.
care-tasks	GET /care-tasks/notifications/pending-counts	Pending task counts by role.
prescriptions	POST /prescriptions	Author a prescription and compute its verification hash.
prescriptions	GET /prescriptions	List prescriptions visible to the caller.
prescriptions	GET /prescriptions/{id}	Retrieve a prescription with its items.
prescriptions	GET /prescriptions/verify/{hash}	Public verification of authenticity without patient PII.
prescriptions	POST /prescriptions/{id}/dispense	Record dispensing by a pharmacist.
prescriptions	POST /prescriptions/{id}/discontinue	Discontinue with a rationale.
prescriptions	GET /prescriptions/adherence/{patient_id}	Adherence summary for a patient.
prescriptions	POST /prescriptions/adherence/log	Record an adherence event.
insurance	POST /insurance/policies	Record a policy for a patient.
insurance	GET /insurance/policies/{patient_id}	Retrieve a patient's policy.
insurance	POST /insurance/pre-auth/estimate	Compute a deterministic coverage estimate.
insurance	POST /insurance/claims	Create a claim record.
insurance	PATCH /insurance/claims/{id}/adjudicate	Record a claim adjudication outcome.
insurance	GET /insurance/claims	List claims within the tenant.
analytics	GET /analytics/overview	Cached hospital-level operational aggregates.
analytics	GET /analytics/departments	Cached department-level aggregates.
agent	POST /agent/chat	Authenticated conversational turn with server-side tool calling.
agent	GET /agent/health	Agent health and provider reachability.
chat	POST /chat/message	Conversational assistant message on the chat surface.

Module	Method and path	Purpose
chat	GET /chat/voice-config	Voice configuration and feature-flag state.
chat	GET /chat/health	Chat subsystem health.

4.7 Authentication and Authorisation Flow

Figure 4.5 shows the authorisation pipeline. A bearer token is first verified with an explicit algorithm whitelist, which prevents the classic algorithm-substitution attack. The token subject is then loaded as a profile and checked for active status, producing a security context that carries the user identifier, the hospital identifier and the role. Role barriers are applied per endpoint as a fast, informative check. The decisive layer, however, is the database: the backend issues the query within a transaction that sets the request identity so that the row-level security policies evaluate the hospital identifier and role of the caller. Cross-tenant access consequently returns an empty result or a not-found response, and the API deliberately returns a not-found status rather than a forbidden status in order to avoid confirming the existence of another tenant's records.

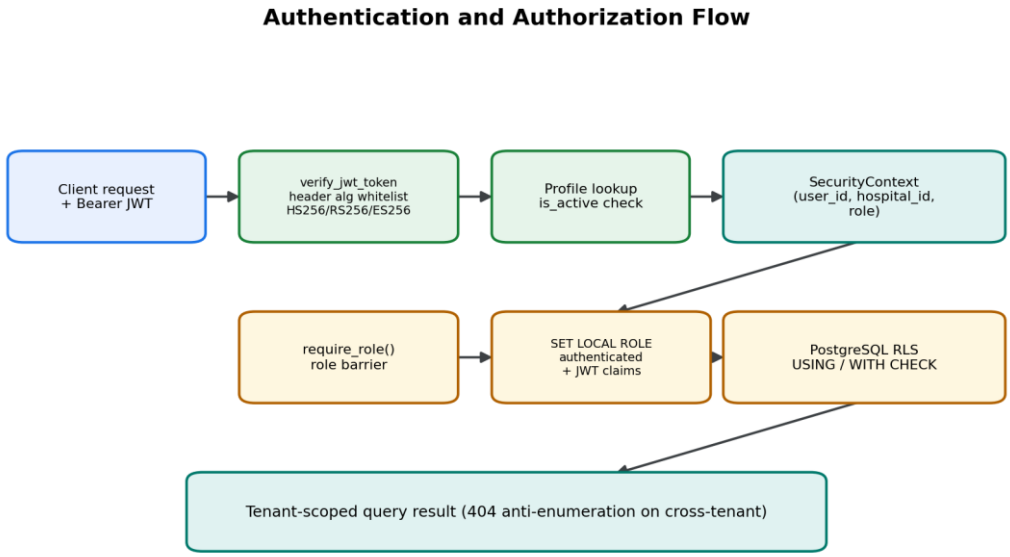


Fig. 4.5 Authentication and authorisation flow from token verification to row-level security

The schema defines five helper functions that the policies use: get_auth_hospital_id(), get_auth_role(), is_hospital_staff(), has_patient_access() and is_patient_record_owner(). Together they express the policy vocabulary in a readable way and keep individual policies short. For example, the appointments select policy grants access when the caller is in the same

hospital and has patient access, which correctly allows patients and caregivers to see their own appointments while preventing them from seeing other patients' appointments.

4.8 Detailed Interaction Designs

4.8.1 Appointment Booking

Figure 4.6 shows the booking sequence. The interesting step is the third message: the insert is protected by a GiST exclusion constraint, so a concurrent overlapping booking fails at the database with an exclusion violation regardless of race conditions in the application. The API maps that violation to a conflict response, and the front end offers alternative slots.

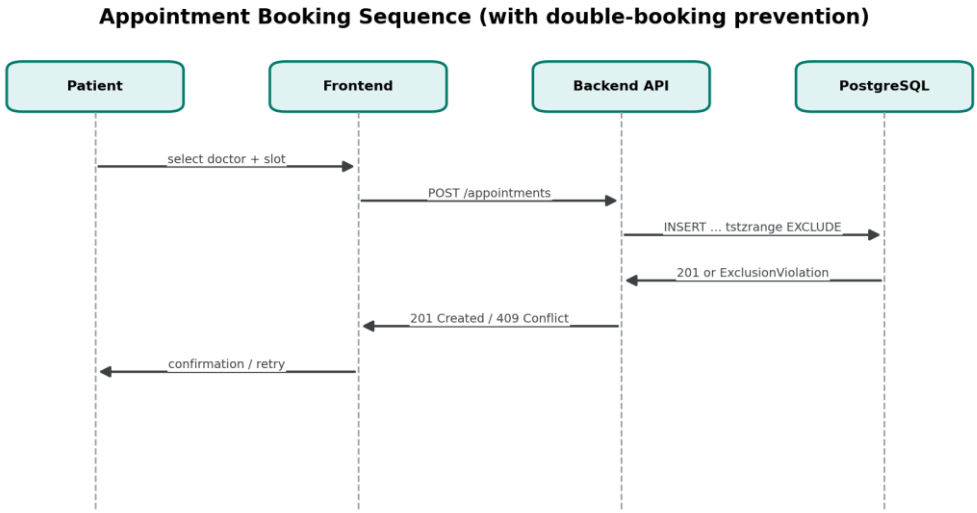


Fig. 4.6 Sequence diagram for appointment booking with double-booking prevention

4.8.2 Document Intake and Verification

Figure 4.7 shows the document pipeline. Uploads are validated for content type and size, stored in private, tenant-isolated storage, then processed for text extraction with an OCR fallback. Candidate entities are extracted and stored as provisional. The pipeline then stops and waits: nothing becomes an authoritative clinical record until a clinician verifies, edits or rejects each entity. This is the concrete realisation of the human-in-the-loop principle for extraction.

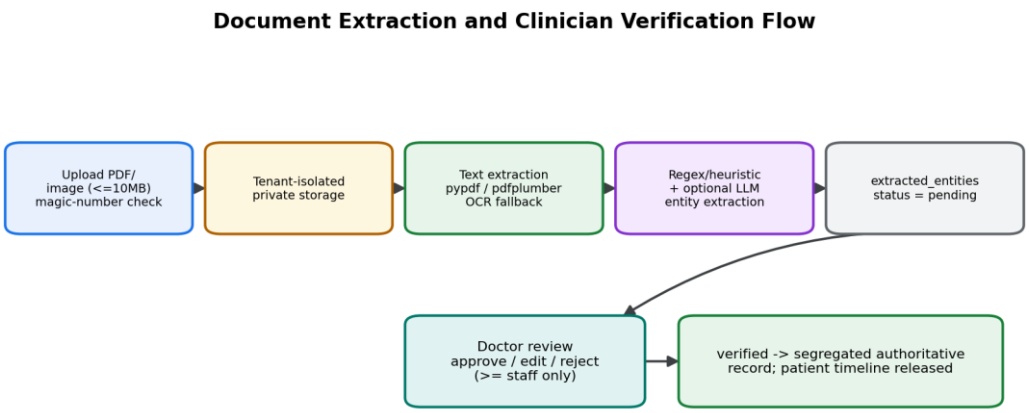


Fig. 4.7 Document extraction and clinician verification flow

4.8.3 Clinical Conflict Detection

Figure 4.8 shows the conflict flow. A scan reads the patient's verified entities and evaluates a curated knowledge base of high-consequence rules covering allergy conflicts, duplicate therapy and drug-drug interactions. Each finding is persisted with its evidence sources, which are recorded in the clinical_conflict_sources table with a role, such as prescribed medication or documented allergy, and a citation string. Findings are tiered by review priority, surfaced as advisory banners, and acknowledged by a clinician with a rationale of at least ten characters. The system never blocks an order; it documents the clinician's decision.

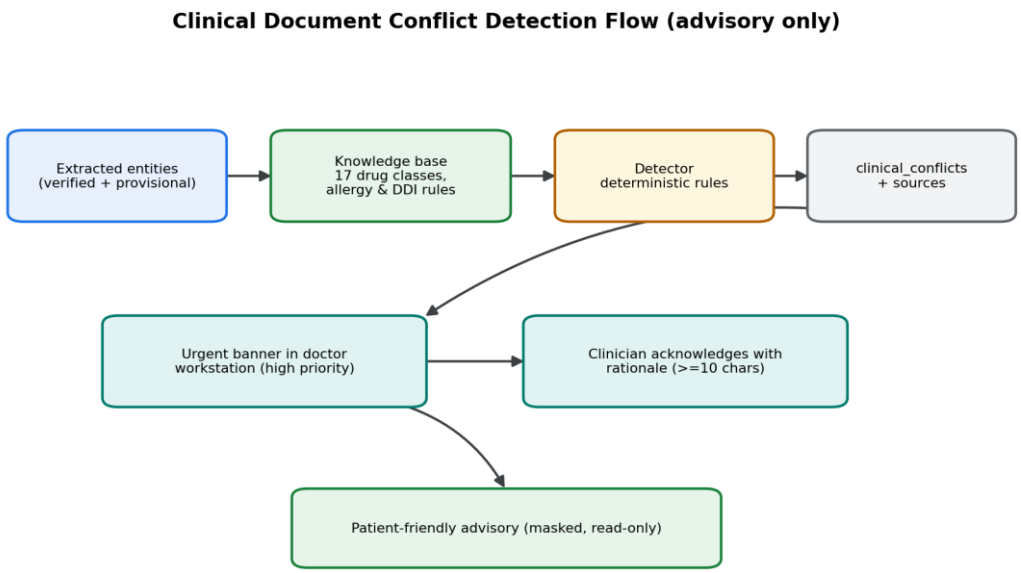


Fig. 4.8 Clinical conflict detection and acknowledgement flow

4.8.4 Care Task Lifecycle

Care tasks model follow-up work that is easily lost. Each task has a type, a priority in the set urgent, high, routine and low, an assigned role, an optional assignee, and a status that moves through pending, claimed, in progress, completed and cancelled. Patient-facing instruction text is gated by a policy that reveals it only to staff or to a caller with patient access, which prevents internal notes from leaking to patients.

4.8.5 Prescription Integrity and Adherence

Figure 4.9 shows the prescription flow. When a prescription is authored, its items are serialised canonically and signed with a keyed hash, producing a verification hash in the form v1 followed by the hexadecimal digest. Any later alteration of the items invalidates the digest. A public endpoint accepts the hash and recomputes the digest in constant time, returning only an authentic or tampered verdict and no patient information, satisfying NFR-06. Adherence is recorded as discrete events with statuses taken, skipped, late and missed, and summarised transparently rather than inferred.

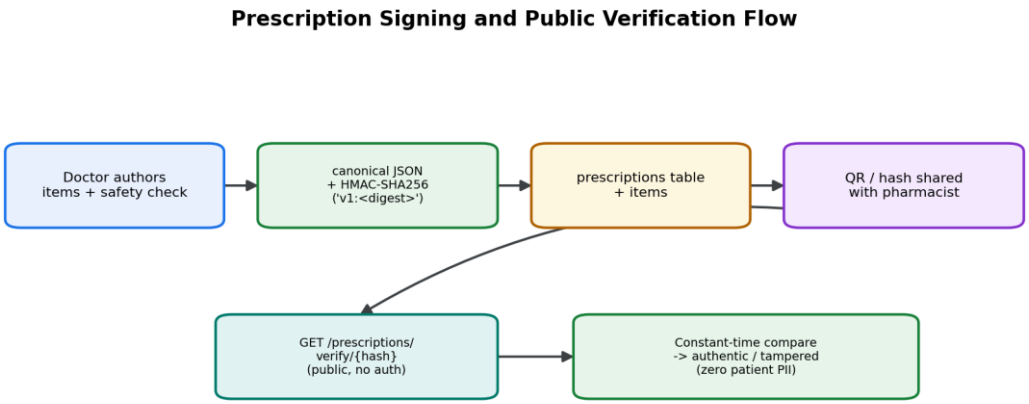


Fig. 4.9 Prescription signing and public verification flow

4.8.6 Insurance Estimation

Figure 4.10 shows the insurance estimation flow. The rule engine applies, in order, a policy completeness and expiry check, an exclusion match, a sub-limit cap, a deductible and a co-pay split. It returns one of three outcomes: insufficient information, review required, or estimate available. When an estimate is available, the response includes the insurer share, the patient share and a full breakdown so that the arithmetic can be audited. The rule version is stored with the claim so that historical estimates remain interpretable after the rules change.

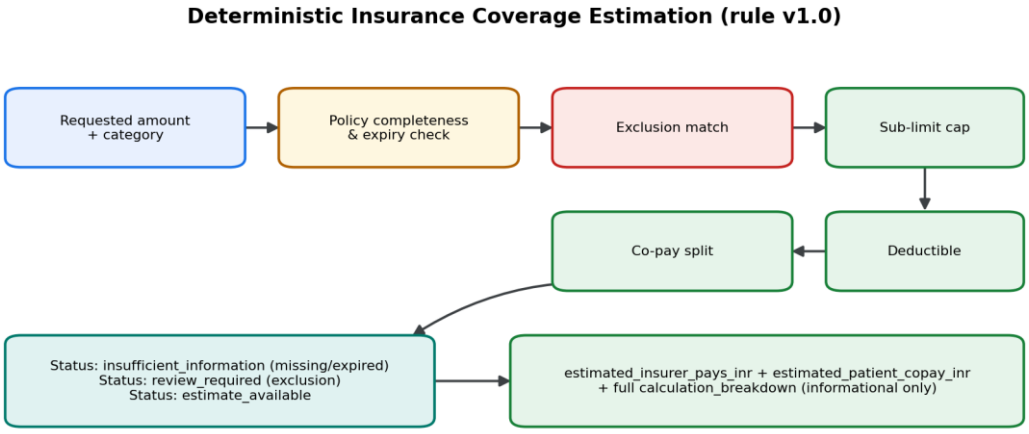


Fig. 4.10 Deterministic insurance coverage estimation flow

4.8.7 Operations Analytics

Figure 4.11 shows the analytics flow. Aggregates are computed with grouped SQL queries and cached in a bounded, tenant-scoped, time-window cache with a sixty-second time to live and a maximum of two hundred and fifty-six entries. Cache keys are composed of the hospital identifier, the time window and the metric name, so that tenants can never observe each other's cached values, and the cache is invalidated for a tenant when that tenant performs a write that affects the aggregates.

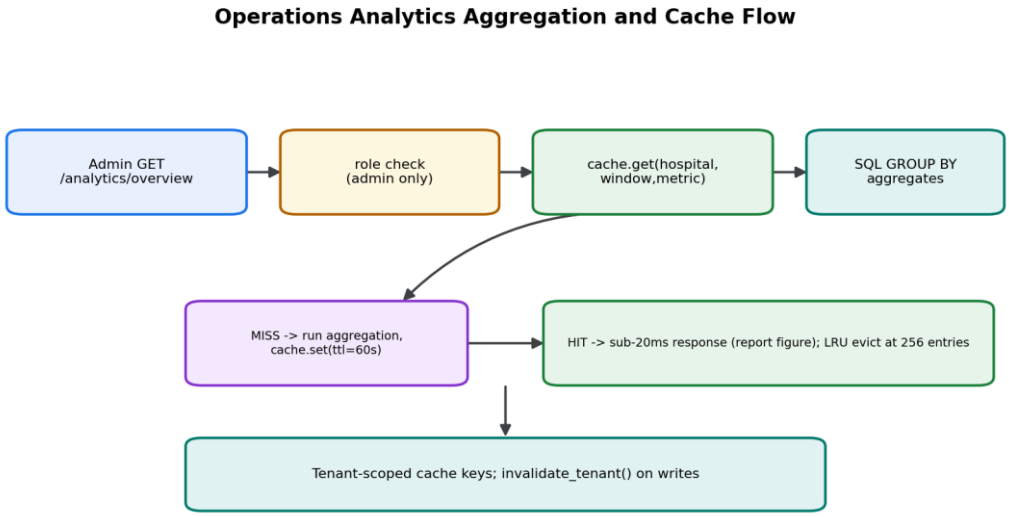


Fig. 4.11 Operations analytics aggregation and caching flow

4.9 Security and Threat Model

The threat model follows the STRIDE categories and maps the mitigations to the OWASP Top Ten classes (OWASP Foundation, 2021). Table 4.3 states the principal threats, the asset at risk, the mitigation as implemented, and the residual risk. Residual risk is stated honestly; where a mitigation is partial, that is recorded.

Table 4.3 Threat model, mitigations and residual risk

Threat	Asset	Mitigation as implemented	Residual risk
Spoofing	Session identity	Bearer token with algorithm whitelist; active-profile check	Token theft remains possible; mitigated by short expiry.
Tampering	Prescription content	Keyed-hash verification digest with version prefix	Key management is out of scope and assumed secure.
Repudiation	Clinical acknowledgements	Audit records with actor, action and timestamp	Audit storage relies on the database tenant boundary.

Threat	Asset	Mitigation as implemented	Residual risk
Information disclosure	Cross-tenant data	Row-level security with composite foreign keys; not-found responses	A misconfigured policy could widen access; policies are reviewed per phase.
Denial of service	Provider and API availability	Circuit breaker with deterministic fallback	A sustained provider outage degrades assistant quality but not safety.
Elevation of privilege	Role field	Privilege-protection trigger; role barriers per endpoint	Administrative account compromise remains a risk.
Injection	Database	Parameterised queries through a typed client	No raw string concatenation is used.
Insecure design	Order-sensitive flows	Advisory-only outputs; mandatory verification step	Human error is inherent and is mitigated, not eliminated.
Prompt injection (assistant)	Assistant behaviour	Operational facts only via authenticated tools; no memory answers	Model output remains untrusted and is presented as guidance.

4.10 Privacy and Compliance Considerations

CareBridge AI processes personal and health data and was designed with privacy-by-design principles in mind. Data is scoped to a single hospital tenant at the database layer; the public verification endpoint exposes only a verdict and no personal information; document files are served through a mediating server call rather than a public link; and caregiver access is explicit, verifiable and revocable. The design is informed by the requirements of the Health Insurance Portability and Accountability Act in the United States and the Digital Personal Data Protection Act, 2023 in India, but the project does not claim formal certification or compliance, and no audit has been performed. This limitation is reiterated in Chapter 10.

4.11 Proposed CSCM Integration Design

Figure 4.12 shows the proposed integration of the CSCM subsystem, which is Phase 12 and is explicitly not implemented. The design is a read-only adapter that loads a pre-trained checkpoint in evaluation mode, consumes synthetic multi-channel windows, and produces a calibrated anomaly score together with channel attribution. Alerts are advisory, are visible only to staff, and are gated by a cooldown so that a single disturbance does not create an alert storm. The design does not feed the score back into any clinical decision, and the interface text explicitly states that the absence of an alert is not evidence of safety.

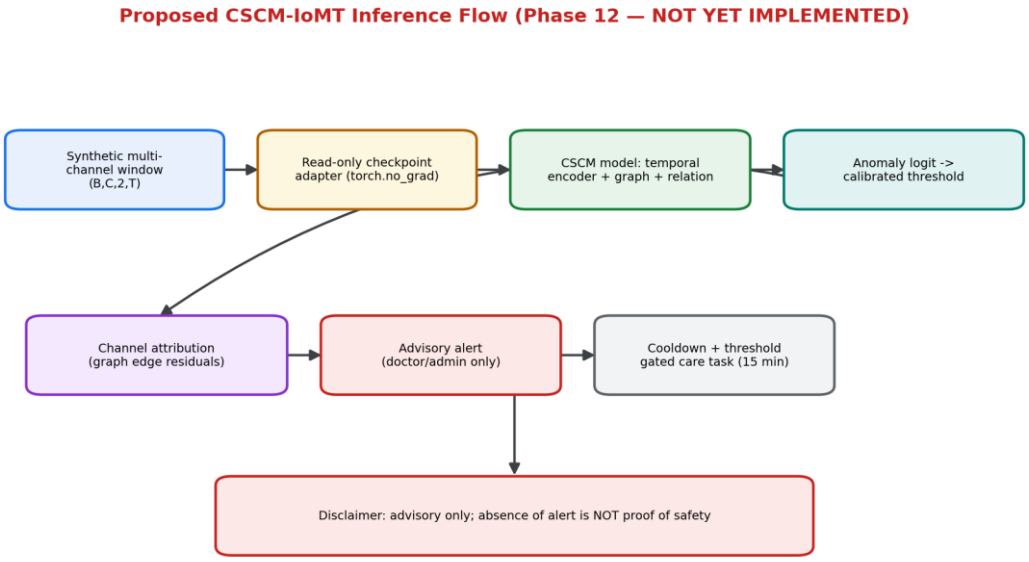


Fig. 4.12 Proposed CSCM-IoMT inference flow (Phase 12, not implemented)

The reason for including the design here, despite its proposed status, is transparency: it shows precisely what integration would entail and where the safety boundaries would lie, while making no claim that any of it exists. The offline evaluation of the model is reported in Chapter 8.

4.12 Phase Dependency Design

Figure 4.13 shows the phase dependency design under which the project was executed. Each phase depends on its predecessors, and each phase terminates in a review gate that must be explicitly authorised before the next phase begins. Phases 0 to 11 are reported complete; Phase 12 is proposed and awaiting authorisation; Phases 13 and 14, covering hardening and release, remain pending.

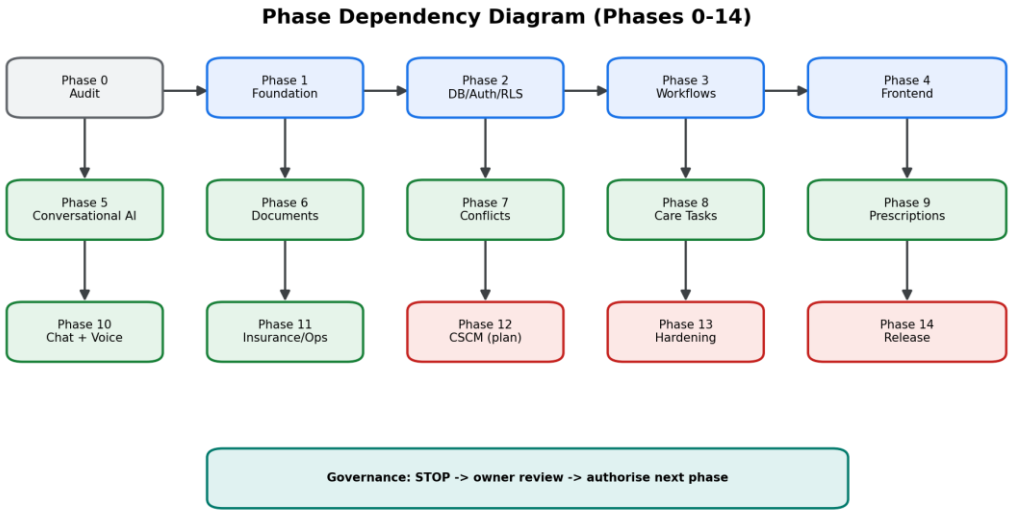


Fig. 4.13 Phase dependency diagram for Phases 0 to 14

4.13 Summary

This chapter defined the design of CareBridge AI: six governing principles, a layered architecture, a deployment topology that runs at zero marginal cost, a seventeen-table tenant-isolated schema, a catalogue of fifty-four API endpoints, an authorisation pipeline that culminates in row-level security, seven detailed interaction designs, a STRIDE-based threat model with honest residual risk, and a clearly labelled design for the proposed Phase 12 integration. Chapter 5 describes how each of these design elements was implemented, phase by phase, and labels the evidence supporting each phase.

CHAPTER 5

IMPLEMENTATION

5.1 Introduction and Delivery Approach

CareBridge AI was delivered through a fifteen-phase plan numbered from Phase 0 to Phase 14. Phases 0 to 11 are the engineering phases and are reported as complete in the project's phase reports. Phase 12 is the research integration phase and remains proposed. Phases 13 and 14 cover hardening and release and remain pending. The phases were executed under a strict gate: each phase produced a plan before work began and a report before the next phase was authorised. This chapter describes what each phase delivered, how it was implemented, and the evidence available to support the claim.

An important discipline applies throughout this chapter. The phrase reported complete means that the phase report asserts completion, and that the assertion is consistent with source code found in the repository. It does not mean that this report's author independently re-ran the phase's tests. Only claims supported by source code, migration SQL or captured test output are described as verifiable from an artefact; claims that rest on a phase report alone are labelled reported.

5.2 Phase 0: Audit and Baseline

Phase 0 produced an audit of the existing repository and a baseline for the project. It inventoried the research assets, the datasets and the tooling, and identified the ten operational weaknesses enumerated in Section 1.2 that became the requirement drivers. The audit is the origin of the evidence hierarchy used throughout this report, because it established which artefacts count as authoritative. Evidence: the audit document and the implementation plan are present in the repository.

5.3 Phase 1: Foundation

Phase 1 established the engineering foundation: repository structure, backend skeleton, configuration management and the coding conventions used by every later phase. Configuration is centralised so that secrets and environment-specific values are read from the environment rather than hard-coded, and the backend is organised so that the routing layer stays thin. Evidence: backend configuration and entry-point source; the repository layout.

5.4 Phase 2: Database, Authentication and Row-Level Security

Phase 2 is the most consequential phase for safety, because it establishes the isolation guarantee. It created the tenant schema (migration 0001), the role enumeration, the row-level security policies, the helper functions used by those policies, and the composite foreign-key discipline described in Section 4.5. Seven tables were created: hospitals, departments, profiles, doctors, patients, patient_accounts and appointments. The migration also installed indexes aligned to the dominant access paths, a trigger protecting privilege-bearing profile fields, and the GiST exclusion constraint that prevents double-booking.

Phase 2 also implemented the authentication pipeline. A bearer token is verified with an explicit algorithm whitelist, the subject is resolved to an active profile, and a security context carrying user, hospital and role is produced for downstream use. Role barriers are applied per endpoint, and the database applies the decisive check. Evidence: migration 0001 SQL including all policies, helper functions and constraints; the security module source.

Table 5.1 Tables created in Phase 2 and their key constraints

Table	Key constraints
hospitals	Primary key on id; unique slug; unique license_number
departments	Composite unique (hospital_id, code); composite unique (hospital_id, id)
profiles	Primary key referencing auth.users; role enumeration; composite unique (hospital_id, id)
doctors	Composite FKs to profiles and departments; unique (profile_id); unique (hospital_id, license_number)
patients	Composite unique (hospital_id, mrn); blood-group and gender checks; composite unique (hospital_id, id)
patient_accounts	Composite FKs to profiles and patients; unique (user_id, patient_id); relationship check
appointments	Check (ends_at > scheduled_at); GiST exclusion preventing doctor double-booking

5.5 Phase 3: Core Workflow Services

Phase 3 implemented the day-to-day workflows: organisation and department retrieval, doctor search and slot generation, patient registration and search, appointment creation with status transitions and rescheduling, and caregiver linkage. Caregiver linkage is modelled in the patient_accounts table with a status that begins at pending_verification and is verified by a responsible party or revoked. A supplementary migration added hospital time zones and a helper function, is_patient_record_owner, used to let a patient see their own account links. Evidence: routers for organization, doctors, patients, appointments and caregiver; migration

0002.

5.6 Phase 4: Front-End Application

Phase 4 delivered the Next.js 16 front end with five role-specific surfaces: a patient portal, a caregiver view, a doctor clinical workstation, a receptionist desk and an administrator console. The doctor workstation is the most safety-critical surface: it presents the exam queue, vitals, allergies and an authorised notes modal, and it is the place where extracted entities are verified and conflicts are acknowledged. The admin console presents capacity, doctors and departments. Evidence: front-end page and component source; role-based navigation.

5.7 Phase 5: Conversational Clinical and Operations Agent

Phase 5 implemented the conversational agent. The design is a server-side tool-calling loop. The model is given a set of tool definitions that correspond to the platform's own APIs; when the user asks an operational question, the loop executes the selected tool under the caller's security context, so that row-level security and role barriers apply exactly as they would for a direct API call. The agent is instructed never to answer operational questions from its own memory. The engine also implements emergency triage detection: inputs that indicate an emergency produce an escalation response with emergency contact guidance. A circuit breaker with a deterministic fallback keeps the assistant available when the provider is unreachable.

The phase test log reports eleven passing tests covering red-flag symptom detection, emergency escalation, doctor search, slot retrieval, booking with double-booking prevention, the two-hour cancellation cut-off, hospital information lookup, circuit-breaker fallback, and three endpoint behaviours. These tests are reported as passing; this report has not independently re-executed them. Evidence: agent engine, client, prompts and tools source; phase report test log (reported).

5.8 Phase 6: Document Intelligence

Phase 6 implemented document intake. The documents module validates uploads, stores them in tenant-isolated private storage, extracts text with a PDF parser and an OCR fallback, and extracts candidate entities into the extracted_entities table with a pending verification status. Migration 0003 created clinical_documents and extracted_entities with full row-level security and indexes on the tenant-patient, tenant-status and uploader access paths. A data-leakage audit

was performed in this phase to check that the extraction pipeline and the notification paths did not expose cross-tenant data. Evidence: migration 0003 SQL; documents router and storage source; phase data-leakage audit document.

5.9 Phase 7: Clinical Conflict Detection

Phase 7 implemented advisory conflict detection. Migration 0004 created clinical_conflicts and clinical_conflict_sources. The conflict types implemented are drug-allergy conflict, duplicate therapy, drug-drug interaction, diagnostic discrepancy, temporal inconsistency and a generic other category. Each conflict carries a review priority in the set high, moderate and low, a clinical summary, a patient-friendly summary, and a resolution status that includes acknowledged_with_rationale. The knowledge base is a curated, auditable set of high-consequence rules rather than an exhaustive pharmacopoeia, which keeps it explainable. Acknowledgement requires a rationale and is attributed to the acknowledging clinician. Evidence: migration 0004 SQL; knowledge-base and detector source.

5.10 Phase 8: Care Task Management

Phase 8 implemented prioritised care tasks. Migration 0005 created the care_tasks table with eight task types, a four-level priority, a five-state status lifecycle, an assigned role, an optional assignee, an optional due date, and patient-facing instructions that are policy-gated so that internal notes are not exposed to patients. The router implements create, list, retrieve, claim, complete and cancel, plus pending-count notifications by role. Evidence: migration 0005 SQL; care-tasks router source.

5.11 Phase 9: Prescription Integrity and Adherence

Phase 9 implemented prescriptions with tamper-evidence and adherence tracking. Migration 0006 created prescriptions, prescription_items and medication_adherence_logs. Each prescription stores a verification_hash with a version prefix; the hash is computed over a canonical serialisation of the prescription and its items using a keyed hash, so that any modification changes the digest. A public verification endpoint recomputes and compares the digest in constant time and returns only an authenticity verdict, exposing no patient information. Adherence events are stored with statuses taken, skipped, late and missed, and summarised. Evidence: migration 0006 SQL; cryptographic and adherence source;

prescriptions router.

5.12 Phase 10: Conversational Assistant and Optional Voice

Phase 10 extended the conversational capability to a hospital-aware assistant and an optional voice prototype that remains disabled unless it is explicitly configured. The assistant is grounded in the platform's APIs in exactly the same way as the Phase 5 agent, and it is bounded so that it cannot bypass the platform's clinical safeguards: it reports operational facts, it does not author clinical decisions, and it defers order-sensitive questions to the corresponding clinician-facing surface. The voice configuration endpoint exposes the feature-flag state so that the front end can hide the voice controls when the feature is off. Evidence: chat router source; phase report (reported).

5.13 Phase 11: Insurance Estimation and Operations Analytics

Phase 11 implemented the finance and operations services. Migration 0007 created patient_insurance_policies and insurance_claims. The insurance module stores policy parameters including sum insured, remaining balance, co-pay percentage, deductible and a JSON list of exclusions, and it computes estimates through a deterministic rule engine tagged with a rule version. Each estimate returns one of three outcomes and, when an estimate is available, a full breakdown. The analytics module computes hospital-level and department-level aggregates through grouped queries and serves them through a bounded, tenant-scoped, time-window cache with a sixty-second time to live and a maximum of two hundred and fifty-six entries.

The Phase 11 report claims that thirteen phase tests passed, that the combined Phase 7 to Phase 11 regression suite passed sixty-one of sixty-one tests, that the front-end lint completed cleanly, and that the front-end production build generated sixteen routes. These figures are reported by the phase report and have not been independently re-executed for this report; they are quoted here as reported evidence in accordance with the evidence hierarchy.

Table 5.2 Phase 11 reported verification figures

Metric	Reported value	Verification status
Phase 11 tests	13 of 13 passed	Reported in phase report; not independently re-run
Combined Phase 7-11 regression	61 of 61 passed	Reported in phase report; not independently re-run
Front-end lint	Clean	Reported in phase report

Metric	Reported value	Verification status
Front-end production build	16 routes generated	Reported in phase report

5.14 Phase 12: CSCM-IoMT Integration (Proposed)

Phase 12 is the integration of the Cross-Sensor Consistency Model into the platform and remains proposed. A detailed plan exists and records three design decisions: a direct read-only inference adapter that loads pre-trained checkpoints in evaluation mode; an interactive multi-channel synthetic stream drawn from preserved test sets with toggleable attack scenarios; and an advisory alert with a deduplicated, cooled-down care task for biomedical engineering or clinical review. Seven safety clarifications were recorded in response to a safety review. None of this has been implemented in the platform, and no approval to implement it has been recorded. The phase therefore remains proposed, and the model's offline evaluation reported in Chapter 8 must not be read as evidence that the platform detects attacks.

5.15 Phase 13: Hardening (Pending)

Phase 13 is intended to cover security hardening, performance tuning and operability improvements. No plan, implementation or report exists for it. It is therefore recorded as not started.

5.16 Phase 14: Release (Pending)

Phase 14 is intended to cover release preparation, documentation finalisation and demonstration. No plan, implementation or report exists for it. It is recorded as not started.

5.17 Phase Status Summary

Table 5.3 summarises the status of every phase together with the strongest available evidence and the verification label used in this report.

Table 5.3 Phase status, evidence and verification label

Phase	Scope	Status	Strongest evidence	Label
0	Audit and baseline	Reported complete	Audit document	Reported
1	Foundation	Reported complete	Configuration and entry-point source	Verifiable
2	Database, auth, RLS	Reported complete	Migration 0001 SQL, security source	Verifiable
3	Core workflows	Reported complete	Router source, migration 0002	Verifiable
4	Front-end application	Reported complete	Front-end page source	Verifiable

Phase	Scope	Status	Strongest evidence	Label
5	Conversational agent	Reported complete	Agent source, phase test log	Reported
6	Document intelligence	Reported complete	Router and storage source, migration 0003	Verifiable
7	Conflict detection	Reported complete	Knowledge-base source, migration 0004	Verifiable
8	Care tasks	Reported complete	Router source, migration 0005	Verifiable
9	Prescriptions	Reported complete	Crypto source, migration 0006	Verifiable
10	Assistant and voice	Reported complete	Chat router source	Reported
11	Insurance and analytics	Reported complete	Rules and cache source, migration 0007	Reported
12	CSCM integration	Proposed	Plan document only	Not implemented
13	Hardening	Not started	None	Not started
14	Release	Not started	None	Not started

The label column is deliberately conservative. Phases whose behaviour is defined by source code that can be read directly are labelled verifiable, meaning that a reader can confirm the claim from the repository. Phases whose evidence includes test results are labelled reported, because the test logs come from the phase reports and were not re-executed for this report. Phase 12 is labelled not implemented, and Phases 13 and 14 are labelled not started.

5.18 Cross-Cutting Implementation Concerns

Several implementation concerns cut across phases and are described here once rather than repeated in each phase. Configuration is read from the environment so that no secret or environment-specific value is committed, and the same practice applies to the language-model credential and to the database connection. Error handling follows the platform's consistent problem-document convention so that a failure is machine-readable and does not leak internal state, as required by NFR-07. Auditing is centralised in a single service so that a security-relevant or clinical-coordination action emits a structured record regardless of which module performed it, satisfying NFR-03.

Database access is performed through a single connection layer that applies the caller's tenant context, so that row-level security has the information it needs on every statement. The front end talks to the backend through typed API clients, which is why a clean production build is a meaningful type-contract check rather than a cosmetic ritual. The research component is kept deliberately separate from the running platform: the models, the preprocessing and the evaluation live outside the services, and Phase 12 is the only place where the two would be

connected, which is why Phase 12 must remain proposed until it is approved and implemented.

These cross-cutting choices are the reason the per-phase descriptions above can stay short. Each phase added capability without having to re-decide how configuration, errors, auditing, tenant context or type contracts would be handled.

5.19 Summary

This chapter described the implementation of Phases 0 to 14 and labelled the evidence supporting each phase. Phases 0 to 11 are reported complete and are consistent with the source code in the repository; Phase 12 is proposed; Phases 13 and 14 are not started. Chapter 6 presents the algorithms that the implementation and the research component use.

CHAPTER 6

ALGORITHMS AND PSEUDOCODE

6.1 Introduction

This chapter records the algorithms that underlie the implemented platform and the proposed research component. Each algorithm is presented as numbered pseudocode with a short explanation, and the chapter closes with a complexity analysis. The algorithms for the platform are expressed at the level of the domain logic rather than the syntax of any particular framework, so that they can be checked against the implementation independently. The algorithms for the research component are expressed at the level of the mathematical formulation that the training code implements.

6.2 Notational Conventions

Sequences are indexed from one. The symbol X denotes a multivariate sensing window with shape (T, C), where T is the window length and C is the number of channels. The symbol z_i denotes the temporal embedding of channel i. The symbol A denotes the adjacency of the physiological channel graph. The operator that concatenates two vectors is written as a comma-separated list, and the element-wise product is written with the symbol for the Hadamard product. The absolute difference of two vectors is written abs(u - v) to keep the notation unambiguous in plain text.

6.3 Tenant-Scoped Authorisation

Algorithm 6.1 is the heart of the isolation guarantee. The decisive step is that the database statement is executed within a transaction whose request identity is set to the caller, so that the row-level security policies evaluate the caller's hospital identifier and role. The application-layer role check is retained as a fast, informative barrier, but correctness does not depend on it.

```
Algorithm 6.1 Tenant-Scoped Authorisation
Input:  HTTP request R with bearer token Tok
Output: Authorised result, or an error response
1:  if Tok is absent or malformed then return 401 Unauthorised
2:  alg <- header.alg(Tok)
3:  if alg is not in {HS256, RS256, ES256} then return 401
4:  claims <- verify(Tok, alg)              # signature and expiry
5:  profile <- select profile where id = claims.sub
6:  if profile is null or profile.is_active = false then return 401
7:  ctx <- (user_id = profile.id, hospital_id = profile.hospital_id,
       role = profile.role)
```

```
8:  if endpoint requires role set S and ctx.role is not in S then return 403
9:  begin transaction
10: set local request.jwt.claims <- claims   # enables RLS evaluation
11: rows <- execute domain query in the caller identity
12: commit transaction
13: if rows is empty for a single-record read then return 404   # anti-enumeration
14: return rows
```

6.4 Double-Booking Prevention

Algorithm 6.2 relies on the storage engine rather than on a check-then-insert sequence in the application, because a check-then-insert sequence is racy under concurrency. The exclusion constraint over the doctor identifier and the half-open timestamp range makes an overlapping booking impossible even when two requests arrive simultaneously.

```
Algorithm 6.2 Booking an Appointment Without Double-Booking
Input:  doctor_id, patient_id, scheduled_at, duration
Output: Created appointment, or 409 Conflict
1:  ends_at <- scheduled_at + duration
2:  if ends_at <= scheduled_at then return 422 InvalidRequest
3:  attempt insert into appointments (doctor_id, patient_id,
       scheduled_at, ends_at, status = 'scheduled')
4:  if the insert violates the exclusion constraint on
       (doctor_id, tstzrange(scheduled_at, ends_at)) then
5:      return 409 Conflict with the set of overlapping intervals
6:  else
7:      return 201 Created with the appointment representation
8:  end if
```

6.5 Document Extraction and Clinician Verification

Algorithm 6.3 performs extraction but deliberately stops short of accepting the result. Extracted entities are persisted with the status pending and only become authoritative when a clinician verifies or edits them. Step 9 is the safety-critical step: a rejected entity never enters the verified set, and an edited entity is stored with the edited value and the identity of the editing clinician.

```
Algorithm 6.3 Extract-then-Verify Document Pipeline
Input:  uploaded file F, patient_id, document_type
Output: Provisional entities pending clinician verification
1:  validate that F.mime_type is allowed and F.size <= limit
2:  path <- store(F) in tenant-isolated private storage
3:  doc <- insert clinical_documents (patient_id, path, ocr_status='pending')
4:  text <- extract_text(F)              # parser with OCR fallback
5:  doc.ocr_status <- 'completed'
6:  candidates <- extract_entities(text)   # categories and confidence
7:  for each candidate c in candidates do
8:      insert extracted_entities (doc.id, c.category, c.data,
           c.confidence, verification_status='pending')
9:  end for
10: await clinician decision for each entity:
11:     approve -> verification_status='verified'
12:     correct -> verification_status='edited' with corrected data
13:     reject  -> verification_status='rejected'
14: only 'verified' and 'edited' entities are authoritative
```

6.6 Clinical Conflict Detection

Algorithm 6.4 evaluates a curated set of high-consequence rules against the patient's verified entities and records each finding with its supporting evidence. The algorithm is advisory: it never modifies an order. It returns findings tiered by priority and each finding cites the sources that produced it, which is what allows a clinician to make an informed decision quickly.

```
Algorithm 6.4 Advisory Conflict Scan
Input:  patient_id
Output: Set of persisted conflicts with evidence
1:  V <- verified and edited entities of patient_id
2:  findings <- empty set
3:  for each medication m in V do
4:      for each allergy a in V where a.category = 'allergy' do
5:          if conflicts_with(m, a) then add_finding(drug_allergy, m, a)
6:      end for
7:      for each other medication n in V, n not equal to m do
8:          if therapeutic_class(m) = therapeutic_class(n) then
9:              add_finding(duplicate_therapy, m, n)
10:         if interaction(m, n) is in knowledge_base then
11:             add_finding(drug_drug_interaction, m, n, severity)
12:     end for
13: end for
14: for each finding f do
15:     persist conflict with review_priority(f)
16:     persist each source of f in clinical_conflict_sources
17: end for
18: return findings   # advisory only; no order is modified
```

6.7 Prescription Signing and Verification

Algorithm 6.5 gives the prescription integrity mechanism. Signing uses a keyed hash over a canonical serialisation, and the stored value carries a version prefix so that the scheme can be evolved without ambiguity. Verification is public and compares digests in constant time, exposing only a verdict. The canonical serialisation is essential: two different serialisations of the same content must never produce different digests, so the algorithm fixes the field order and the numeric formatting.

```
Algorithm 6.5 Prescription Signing and Public Verification
Input:  prescription header P and item set I, secret key K
Output: Signing: a versioned digest. Verification: a verdict.
1:  S <- canonical_json(P, ordered_fields(I))
2:  d <- HMAC_SHA256(K, S)
3:  verification_hash <- 'v1:' + hex(d)
4:  store verification_hash with the prescription and items
        ---- verification (public, no PII) ----
5:  given verification_hash from a pharmacist
6:  parse prefix 'v1'; if prefix is unknown then return unsupported
7:  recompute d' over the stored canonical serialisation
8:  if constant_time_equal(d, d') then return authentic
9:  else return tampered
10: never return patient identifiers or prescription content
```

6.8 Insurance Estimation

Algorithm 6.6 is a deterministic rule pipeline. Its most important property is that it can decline to produce a number: when the policy is incomplete or expired it returns an explicit insufficiency outcome, and when an exclusion matches it returns a review outcome. Only when the checkpoints pass does it compute an estimate, and it always returns the breakdown so that the arithmetic can be audited.

```
Algorithm 6.6 Deterministic Coverage Estimation (rule v1.0)
Input:  policy, requested_amount, category
Output: status and, when available, an estimate with a breakdown
1:  if policy is missing or any required field is null then
2:      return insufficient_information
3:  if today is not within the policy validity window then
4:      return insufficient_information
5:  if category matches an exclusion in policy.exclusions then
6:      return review_required with the matched exclusion
7:  eligible <- min(requested_amount, policy.sum_insured,
       policy.remaining_balance, sub_limit(category))
8:  base <- max(0, eligible - policy.deductible)
9:  insurer_share <- base * (1 - policy.copay_percentage / 100)
10: patient_share <- requested_amount - insurer_share
11: return estimate_available with insurer_share, patient_share
        and the full intermediate breakdown, tagged rule_version = v1.0
```

6.9 Cached Analytics

Algorithm 6.7 defines the analytics cache. Tenant isolation is preserved by construction, because the cache key includes the hospital identifier; a value computed for one hospital can therefore never be returned to another. The time to live bounds staleness, the entry cap bounds memory, and explicit invalidation keeps the aggregates correct after a write.

```
Algorithm 6.7 Bounded Tenant-Scoped Analytics Cache
Input:  hospital_id, time_window, metric
Output: Aggregated metric value
1:  key <- hospital_id + '::' + time_window + '::' + metric
2:  entry <- store.get(key)
3:  if entry exists and now - entry.stored_at < ttl then
4:      return entry.value                      # cache hit, no query
5:  value <- run grouped SQL aggregation scoped to hospital_id
6:  if number of entries >= max_entries then evict the oldest entry
7:  store.put(key, (now, value))
8:  return value
9:  on any write that affects aggregates: invalidate_all(hospital_id)
```

6.10 CSCM Problem Formulation

The research component addresses coordinated multi-channel false data injection in IoMT. Let X denote a window of a multivariate physiological recording with shape (T, C), where T is the number of time steps and C is the number of channels. Let y be the binary window label, where

y equals one if the window contains at least one injected attack sample in any channel. The detection problem is to learn a scoring function that assigns a high score to attacked windows and a low score to clean windows.

$$X \in \mathbb{R}^{T \times C}, \quad y \in \{0, 1\} \tag{6.1}$$

The threat model is the coordination condition. An adversary selects a set of channels and perturbs them such that every perturbed channel remains within its marginal plausibility bounds, so that a univariate detector observes nothing unusual. Formally, for each perturbed channel c the perturbed signal satisfies the marginal bound constraint, yet the joint state violates the consistency relations that physiology imposes between channels.

$$\text{for all perturbed } c: \quad x'_c \text{ is within } [lo_c, hi_c] \tag{6.2}$$

The architecture is a sequence of five stages. First, a shared temporal encoder maps each channel to an embedding. Second, a physiology-informed graph defines which channel pairs are compared. Third, a relation network computes a message for each edge. Fourth, messages are aggregated at each node by permutation-invariant summation. Fifth, a pooled representation is classified to produce the window probability. The three evaluated models differ only in the fusion stage, which is what makes the comparison a clean ablation.

$$z_i = \text{Encoder}(x_i), \quad z_i \in \mathbb{R}^{256}, \quad \text{for } i = 1 \text{ to } C \tag{6.3}$$

$$\text{Model A:} \quad h = \text{MLP}(\text{concat}(z_1, \ldots, z_C)) \tag{6.4}$$

$$\text{Model B:} \quad r_{ij} = \text{concat}(z_i, z_j) \tag{6.5}$$

$$\text{Model C:} \quad r_{ij} = \text{concat}(z_i, z_j, \text{abs}(z_i - z_j), z_i \odot z_j) \tag{6.6}$$

$$m_{ij} = \text{RelationMLP}(r_{ij}), \quad \text{agg}_j = \sum_i m_{ij} \tag{6.7}$$

$$\hat{y} = \text{sigmoid}(W \cdot \text{pool}(\text{agg}) + b) \tag{6.8}$$

6.11 CSCM Training and Evaluation Algorithm

Algorithm 6.8 describes window construction and label assignment. Algorithm 6.9 describes training, which is implemented in PyTorch (Paszke et al., 2019). Two controls are essential and are both implemented: normalisation statistics are computed on the training split alone to prevent leakage, and the decision threshold is calibrated on the validation split and then frozen before the test split is touched. Algorithm 6.10 describes the hierarchical paired bootstrap and

the McNemar test used to assess whether an observed difference is robust.

```
Algorithm 6.8 Window Construction and Attack Labelling
Input:  raw records split by subject into train, validation and test
Output: windows X with labels y and per-sample attack masks
1:  align all channels of a record to a shared time axis
2:  resample to a uniform step; forward-fill short gaps
3:  compute channel means and standard deviations on TRAIN only
4:  apply the frozen scaling to train, validation and test
5:  for each record in each split do
6:      inject synthetic attacks (additive, multiplicative, drift)
7:      over a subset of channels, respecting plausibility bounds
8:      for each rolling window of length T = 120 do
9:          y <- 1 if the attack mask has any positive sample else 0
10:     emit (window, y)
11: end for
Algorithm 6.9 CSCM Training with Frozen Threshold
Input:  training windows of models A, B and C
Output: trained weights and a frozen decision threshold
1:  for each model M in {A, B, C} do
2:      initialise weights with the current seed
3:      repeat
4:          predict y_hat for a training mini-batch
5:          L <- weighted_BCE(y, y_hat)     # class-weighted
6:          update weights with Adam
7:          evaluate AUROC on the validation split
8:      until validation AUROC stagnates (early stopping)
9:      tau <- argmax over t of validation F1(t)   # freeze threshold
10:     evaluate on the test split using tau
11: end for
Algorithm 6.10 Robust Comparison of Two Models
Input:  paired test predictions of models P and Q
Output: 95% bootstrap intervals and a McNemar p-value
1:  for b <- 1 to 1000 do
2:      resample subjects with replacement
3:      compute delta(b) = Metric(P) - Metric(Q) on the resample
4:  end for
5:  report the 2.5 and 97.5 percentiles of delta as the 95% interval
6:  build the 2x2 table of paired binary disagreements (b, c)
7:  McNemar statistic <- (abs(b - c) - 1)^2 / (b + c)
8:  p-value <- chi_square_survival(statistic, df = 1)
9:  a difference is robust only if the interval excludes zero and
10:     the McNemar p-value is below 0.05
```

6.12 Complexity Analysis

Table 6.1 summarises the cost of each algorithm in terms of the dominant input size. Both the platform algorithms and the research algorithms are tractable on commodity hardware, which is what allows the project to meet its zero-marginal-cost constraint while training thousands of model parameter combinations across three seeds.

Table 6.1 Complexity of the principal algorithms

Algorithm	Dominant cost	Notes
6.1 Tenant authorisation	O(1) index lookup plus query cost	Token verification is constant time relative to data size

Algorithm	Dominant cost	Notes
6.2 Booking	O(log n) index probe for the exclusion check	GiST index probe; conflict detection in the engine
6.3 Extraction	O(len(text)) per document	Linear in extracted text length; OCR dominates wall time
6.4 Conflict scan	O(m squared) over verified medications	m is small in practice (typically below 20)
6.5 Signing	O(S) in the serialised size	One keyed hash; verification is the same cost
6.6 Insurance rule	O(e) over exclusions	Deterministic and constant per request
6.7 Analytics cache	O(1) on hit; aggregation cost on miss	Cache converts repeated reads into constant time
6.8 Windowing	O(N x T x C) over the corpus	One pass per split; masks are T x C
6.9 CSCM training	O(C squared x epochs) message evaluations	C is small (a handful of channels)
6.10 Bootstrap	O(B x subjects) metric evaluations	B = 1000 resamples per comparison

6.13 Summary

This chapter specified ten algorithms with pseudocode and complexity analysis: seven for the platform and three for the research component. The platform algorithms embody the safety principles of mandatory verification, storage-enforced correctness and structural tenant isolation. The research algorithms embody leakage-free evaluation and statistically disciplined comparison. Chapter 7 describes how the implementation and the research were tested.

CHAPTER 7

TESTING AND EVALUATION

7.1 Introduction

This chapter describes how CareBridge AI was tested and how the research component was evaluated. It distinguishes clearly between three kinds of evidence: automated tests that the phase reports say were executed, artefacts such as source code and migration SQL that a reader can inspect directly, and the offline research evaluation whose results are reported in Chapter 8. The chapter concludes with an honest statement of what has not been verified.

7.2 Testing Strategy

Testing was performed in five layers. The first layer is unit-level testing of pure domain functions, such as the insurance rule engine and the prescription digest, which can be tested without a database. The second layer is integration testing of routers against a test database, which is where the row-level security behaviour and the exclusion constraint are exercised. The third layer is the phase test suite: each phase contributed tests that were executed at the end of the phase and before the gate review. The fourth layer is regression testing, in which the accumulated phase tests are re-run together. The fifth layer is build and static verification of the front end, which consists of lint and a production build.

Two deliberate omissions should be acknowledged. First, there is no automated browser end-to-end suite; front-end behaviour was verified by the production build and by manual inspection. Second, there is no load or soak testing; the cache and query design were reasoned about for performance but not stress-tested. Both omissions are recorded as limitations.

7.3 Test Environment

Tests were executed in a Windows development environment with a Python 3.13 interpreter and Node.js 20 or later. The backend tests run against a PostgreSQL instance with the migrations applied. Because the tests were executed within the project's own environment, their logs are reported as phase output rather than as independently reproducible results in this report; reproducing them requires the test database described in the setup document.

7.4 Backend Test Suites

The repository contains phase-specific test modules, including an agent test module from Phase 5, a conflicts test module from Phase 7, a care-tasks test module from Phase 8, a prescriptions test module from Phase 9 and an insurance and analytics test module from Phase 11. Table 7.1 lists the reported test outcomes. The Phase 5 agent suite is reported as eleven passing tests with named cases; the Phase 11 report states that its own thirteen tests passed and that, in combination with the Phase 7, 8, 9 and 10 suites, sixty-one of sixty-one tests passed.

Table 7.1 Reported backend test outcomes

Suite	Reported cases	Reported outcome	Verification status
Phase 5 agent tests	11 named cases	11 passed	Reported; test module present in repository
Phase 7 conflict tests	Included in combined run	Passing in combined run	Reported
Phase 8 care-task tests	Included in combined run	Passing in combined run	Reported
Phase 9 prescription tests	Included in combined run	Passing in combined run	Reported
Phase 10 chat tests	Included in combined run	Passing in combined run	Reported
Phase 11 insurance and analytics tests	13 cases	13 passed	Reported
Combined Phase 7 to 11 regression	61 cases	61 passed	Reported

The Phase 5 agent test names are informative because they show the breadth of behaviour that was checked: detection of red-flag symptoms, emergency escalation, doctor search, slot retrieval, booking with double-booking prevention, the two-hour cancellation cut-off, hospital information lookup, offline fallback through the circuit breaker, and three endpoint behaviours for health, emergency chat and live-or-fallback chat. The names correspond to observable behaviours rather than to implementation details, which is good practice.

7.5 Front-End Static and Build Verification

The front-end verification consisted of a lint pass and a production build. The Phase 11 report states that lint completed cleanly and that the production build generated sixteen routes. These figures are reported. The build is a meaningful check because it performs compilation and type checking of the entire application, so a clean build provides reasonable evidence that the type contracts between the pages and the API clients are consistent.

7.6 Security Testing

Security testing focused on the isolation guarantee and on the authorisation path. Three classes of check were performed. First, cross-tenant access was tested by attempting to read a record

belonging to another hospital and confirming that the response is a not-found rather than the record. Second, role enforcement was tested by attempting privileged operations with an unprivileged role and confirming that the request is rejected before data access. Third, the authorisation algorithm was reviewed for the algorithm-substitution vulnerability and confirmed to use an explicit whitelist.

Table 7.2 Security checks and their basis

Check	Method	Result basis
Cross-tenant read prevention	Query as tenant B for a tenant A record	Row-level security policy in migration SQL; not-found response
Cross-tenant integrity	Composite foreign keys on (hospital_id, id)	Database constraints in migration SQL
Role enforcement	Invoke a privileged endpoint with an unprivileged role	Role barriers in router source
Token algorithm substitution	Inspect verification for an algorithm whitelist	Explicit whitelist in security source
Privilege escalation via profile update	Attempt to change role through profile update	Privilege-protection trigger in migration 0001
PII exposure at public verification	Call the public endpoint and inspect the payload	Endpoint returns a verdict only
Upload abuse	Submit an oversized or disallowed file type	Content-type and size validation in the documents module

7.7 Research Evaluation Methodology

The research component was evaluated with the controls described in Chapter 6. Subjects, not windows, were split into training, validation and test partitions, so that a model can never memorise a patient's signature and be tested on that same patient. Normalisation statistics were computed on the training partition only. The decision threshold was calibrated on the validation partition and then frozen. Each model was trained with three seeds, and differences between models were assessed with a hierarchical paired bootstrap that resamples subjects, and with McNemar's test on the paired binary predictions.

These controls matter because the naive alternative, splitting at the window level, would produce optimistically biased results that would not survive scrutiny. The reported results in Chapter 8 show that even with the controls in place the observed differences between models are small, which is itself an important and honest finding.

7.8 Evidence Register

Table 7.3 records the artefacts on which this report relies, together with the claim each artefact supports and the strength of that support. This register is the operational form of the evidence

hierarchy stated at the outset of the report.

Table 7.3 Evidence register

Artefact	Claim supported	Strength
Migration SQL (0001 to 0007)	Schema, constraints, policies and helper functions exist as described	Strong: source of truth
Backend router source	Endpoint catalogue and role barriers exist as described	Strong: source of truth
Core security and audit source	Authentication and auditing behave as described	Strong: source of truth
Prescription crypto source	Tamper-evident signing and constant-time verification	Strong: source of truth
Insurance rules source	Deterministic estimation and rule versioning	Strong: source of truth
Analytics cache source	Bounded, tenant-scoped, time-windowed caching	Strong: source of truth
Agent and chat source	Grounded tool calling and fallback behaviour	Moderate: behaviour-defined; tests reported only
Front-end page source	Five role surfaces exist	Strong: source of truth
Phase reports 1 to 11	Phase completion and reported test counts	Moderate: narrative evidence
Aggregated metrics CSV	Model performance on both datasets	Strong: computed output
Statistical tests CSV	Bootstrap intervals and McNemar results	Strong: computed output
Model checkpoints and metrics JSON	Trained models exist for the reported seeds	Strong: artefacts present

7.9 Verification Gaps and Limitations

The following claims are not independently verified by this report. The test counts reported by the phase reports were not re-executed here. The lint and build outcomes were not re-run here. The hosted language-model behaviour was exercised in the project's own environment with synthetic data, not against a live hospital. No penetration test and no load test were performed. No compliance certification has been obtained. The research results are offline and were obtained on public datasets with synthetic attacks; they do not constitute evidence that a deployed system would detect a real attack. These gaps are restated in Chapter 10.

7.10 Test Case Catalogue

The phase reports record test counts rather than a case-level catalogue, so Table 7.4 reconstructs a representative catalogue from the behaviours that the source code and the phase reports define. Each case is stated as a target requirement, a procedure and an expected result. The catalogue is offered both as a specification of what should be tested and as a description of what the phase tests were reported to cover; the cases were not re-executed for this report, and the evidence column therefore distinguishes cases whose basis is source inspection from

cases whose basis is a phase report.

Table 7.4 Representative test case catalogue

ID	Target	Procedure	Expected result	Evidence basis
TC-01	FR-01, NFR-02	Call an authenticated endpoint with a token whose role lacks permission	Request rejected before any data access	Source inspection of role barriers
TC-02	FR-02, NFR-01	Read a record owned by another hospital using a valid token	Not-found response, no data returned	Row-level security policy in migration SQL
TC-03	FR-06	Insert two appointments for one doctor with overlapping intervals	Second insert rejected by the exclusion constraint	GiST constraint in migration SQL
TC-04	FR-07	Cancel an appointment inside the cut-off window	Cancellation refused with a policy reason	Router source and Phase 7 report
TC-05	FR-11	Approve a provisional extracted entity as a clinician	Entity becomes authoritative and the action is audited	Router and audit source
TC-06	FR-11	Attempt to approve an extracted entity without clinician rights	Request rejected before data access	Role barrier in the documents router
TC-07	FR-12	Verify a record containing a known allergy and a conflicting drug	Conflict raised with its supporting evidence	Knowledge-base source and Phase 7 tests
TC-08	FR-13	Acknowledge a conflict with a rationale	Status becomes acknowledged and the rationale is retained	Conflict router source
TC-09	FR-16, NFR-06	Call the public verification endpoint for a valid prescription	Authenticity verdict returned with no patient identifiers	Crypto source and prescriptions router
TC-10	FR-16	Call public verification after altering a prescription item	Verification fails because the digest changes	Crypto source with constant-time compare
TC-11	FR-18	Request an estimate for a category with no policy record	Insufficient-information outcome rather than a zero estimate	Insurance rules source
TC-12	FR-20	Ask the assistant an operational question	Answer produced by a tool call under the caller's context	Agent source; Phase 5 tests reported
TC-13	FR-22	Simulate a language-model timeout	Deterministic fallback response with no leaked error	Agent client source; Phase 5 tests reported
TC-14	NFR-03	Perform a security-relevant action	Structured audit record emitted	Audit source

The catalogue also exposes deliberate gaps. There is no case for concurrent appointment creation under true parallel load, no browser-level end-to-end case, and no case that exercises the compliance controls against a real regulator's checklist. These are the same

gaps recorded in Section 7.9, and they are restated as limitations in Chapter 10.

7.11 Summary

This chapter described a five-layer testing strategy, reported the test outcomes that the phase reports record, documented the security checks and the research evaluation controls, and stated the verification gaps honestly. Chapter 8 presents the results, beginning with the platform status and then the offline research evaluation.

CHAPTER 8

RESULTS AND DISCUSSION

8.1 Introduction

This chapter presents results in two parts. The first part reports the status of the platform, distinguishing what is verified from what is reported. The second part presents the offline evaluation of the research component in full, including results that are unfavourable to the proposed model, because an honest account of a negative or marginal result is more valuable than an inflated claim.

8.2 Platform Results

The platform result is a set of implemented and documented subsystems. The tenant schema, the row-level security policies, the composite foreign keys, the double-booking exclusion constraint, the document verification workflow, the conflict engine with evidence sources, the care-task lifecycle, the tamper-evident prescription mechanism, the deterministic insurance rule engine and the bounded analytics cache are all present in source artefacts and are described in Chapters 4 and 5. The Phase 11 report additionally states that thirteen phase tests and sixty-one combined regression tests passed, that lint completed cleanly and that the build generated sixteen routes. Those four figures are reported rather than independently re-executed.

Table 8.1 Platform result summary

Result	Status	Basis
Seventeen-table tenant-isolated schema	Implemented	Migration SQL
Fifty-four API endpoints across twelve modules	Implemented	Router source
Row-level security on every tenant table	Implemented	Policy definitions in SQL
Double-booking prevention	Implemented	GiST exclusion constraint
Mandatory clinician verification of extracted entities	Implemented	Status workflow in SQL and routers
Advisory conflict detection with evidence	Implemented	Knowledge base and conflict tables
Prioritised care-task lifecycle	Implemented	Router and table
Tamper-evident prescriptions and adherence	Implemented	Crypto source and tables
Deterministic insurance estimation	Implemented	Rule source with rule version
Bounded tenant-scoped analytics cache	Implemented	Cache source
Reported phase tests, lint and build	Reported	Phase reports

8.3 CSCM Absolute Performance

The research component was evaluated on two public datasets. PhysioNet/CinC 2012 represents a clinical ICU setting with asynchronous, sparse and noisy observations; WESAD represents a wearable setting with high-frequency, synchronised signals. Three models were compared: Model A, a naive multi-channel model that flattens channel embeddings; Model B, a graph-only model that computes pairwise messages over a physiology-informed channel graph; and Model C, the full Cross-Sensor Consistency Model, which augments each edge representation with the absolute difference and the element-wise product of the two channel embeddings. Each model was trained with three random seeds (42, 123 and 2024), and all reported values are means with standard deviations across seeds.

Table 8.2 Absolute performance of the three models across both datasets (mean plus or minus standard deviation over three seeds)

Dataset	Model	F1	AUPRC	AUROC
PhysioNet	A (Naive)	0.8788 plus or minus 0.0019	0.9550 plus or minus 0.0008	0.8946 plus or minus 0.0017
PhysioNet	B (Graph-Only)	0.8863 plus or minus 0.0031	0.9583 plus or minus 0.0012	0.9029 plus or minus 0.0021
PhysioNet	C (CSCM)	0.8827 plus or minus 0.0030	0.9556 plus or minus 0.0019	0.8962 plus or minus 0.0036
WESAD	A (Naive)	0.8509 plus or minus 0.0033	0.8820 plus or minus 0.0087	0.6812 plus or minus 0.0183
WESAD	B (Graph-Only)	0.8489 plus or minus 0.0068	0.8877 plus or minus 0.0035	0.6909 plus or minus 0.0110
WESAD	C (CSCM)	0.8520 plus or minus 0.0008	0.8822 plus or minus 0.0015	0.6806 plus or minus 0.0051

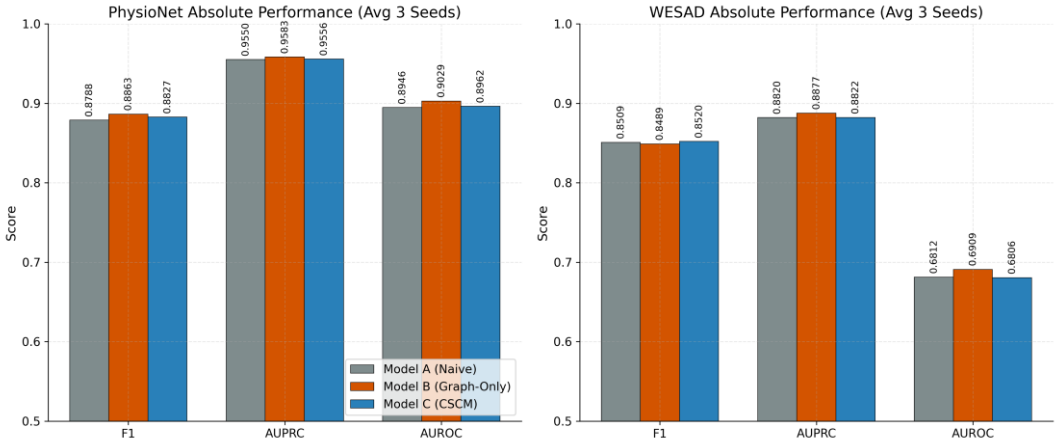


Fig. 8.1 Absolute performance of the three models on both datasets

Three observations follow directly from the table. First, on the clinical PhysioNet dataset the graph-only model achieved the highest value on all three metrics, while the full

CSCM model was slightly behind it. Second, on the wearable WESAD dataset the graph-only model achieved the highest AUPRC and AUROC, while the CSCM model achieved a marginally higher F1. Third, the absolute differences are small in every case, and the WESAD AUROC values, around 0.68 to 0.69, show that the wearable dataset is substantially harder for all three models, most likely because the baseline is already strong and the synchronised signals reduce the discriminative advantage of relational modelling.

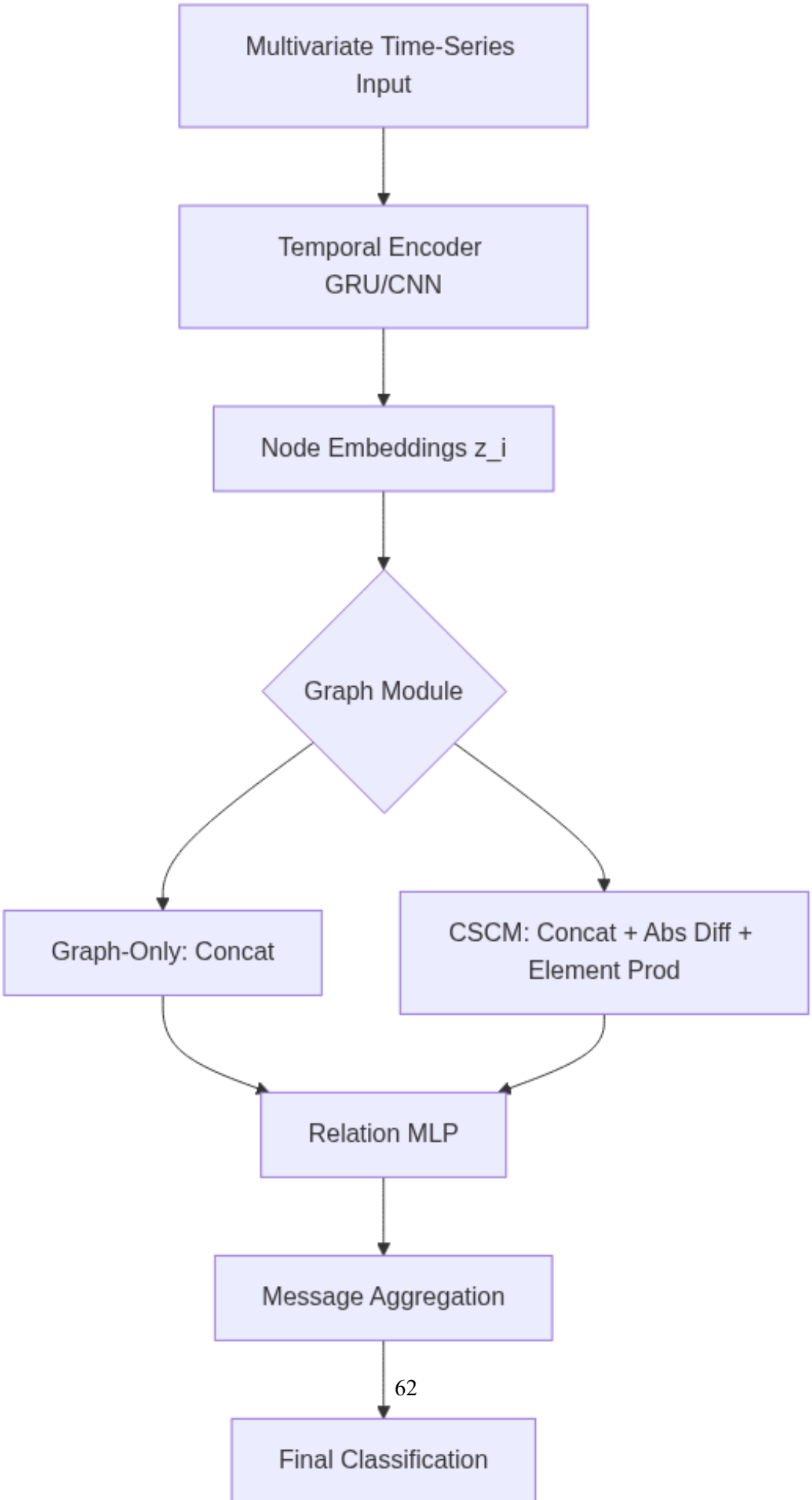

Multivariate Time-Series Input
Temporal Encoder GRU/CNN
Node Embeddings z_i
Graph Module
Graph-Only: Concat
CSCM: Concat + Abs Diff + Element Prod
Relation MLP
Message Aggregation
Final Classification

Fig. 8.2 Architecture of the evaluated multi-channel models

8.4 Architectural Ablation

The ablation isolates the contribution of each architectural mechanism. The difference between Model B and Model A measures the value of physiological graph message passing over naive flat fusion. The difference between Model C and Model B measures the incremental value of the explicitly engineered consistency features over the raw concatenated graph representation.

Table 8.3 Architectural differences averaged over three seeds

Dataset	Comparison	Delta F1	Delta AUPRC	Delta AUROC
PhysioNet	B minus A (Graph vs Naive)	plus 0.0075	plus 0.0033	plus 0.0082
PhysioNet	C minus B (CSCM vs Graph)	minus 0.0035	minus 0.0027	minus 0.0067
WESAD	B minus A (Graph vs Naive)	minus 0.0020	plus 0.0058	plus 0.0097
WESAD	C minus B (CSCM vs Graph)	plus 0.0030	minus 0.0055	minus 0.0103

The pattern is consistent: introducing the physiological graph improves the ranking metrics on both datasets, whereas adding the explicit consistency features does not. On PhysioNet the full CSCM model degraded relative to the graph-only model on all three metrics, and on WESAD it improved F1 but degraded both AUPRC and AUROC. In other words, the hypothesis that manually computing the absolute difference and the element-wise product would accelerate the detection of decoupled signals was not supported by the evidence. A plausible explanation is that the relation network operating on the raw concatenated embeddings is already expressive enough to approximate the required non-linear distance boundaries, so the handcrafted features add capacity without adding information.

8.5 Statistical Robustness

Because neighbouring windows from the same patient are correlated, the significance of the observed differences was assessed with a hierarchical paired bootstrap that resamples subjects rather than windows, and with McNemar's test on the paired binary predictions. Table 8.3 reports selected seed-level results for the F1 metric together with the McNemar p-value.

Table 8.4 Statistical assessment of the architectural differences (selected seed-level results)

Dataset and seed	Comparison	F1 95 percent interval	McNemar p	Interpretation
PhysioNet 42	B minus A	minus 0.0016 to plus 0.0177	0.399	Not significant

Dataset and seed	Comparison	F1 95 percent interval	McNemar p	Interpretation
PhysioNet 42	C minus B	minus 0.0180 to minus 0.0005	0.0198	Significant, in favour of the graph-only model
PhysioNet 123	B minus A	minus 0.0038 to plus 0.0118	0.360	Not significant
PhysioNet 123	C minus B	minus 0.0078 to plus 0.0116	0.508	Not significant
PhysioNet 2024	B minus A	plus 0.0018 to plus 0.0187	0.0003	Significant, in favour of the graph-only model
PhysioNet 2024	C minus B	minus 0.0130 to plus 0.0060	0.443	Not significant
WESAD 42	C minus B	minus 0.0046 to 0.0000	1.000	Not significant
WESAD 123	B minus A	minus 0.0118 to minus 0.0029	1.000	Not significant
WESAD 2024	B minus A	0.0000 to 0.0000	1.000	Not significant

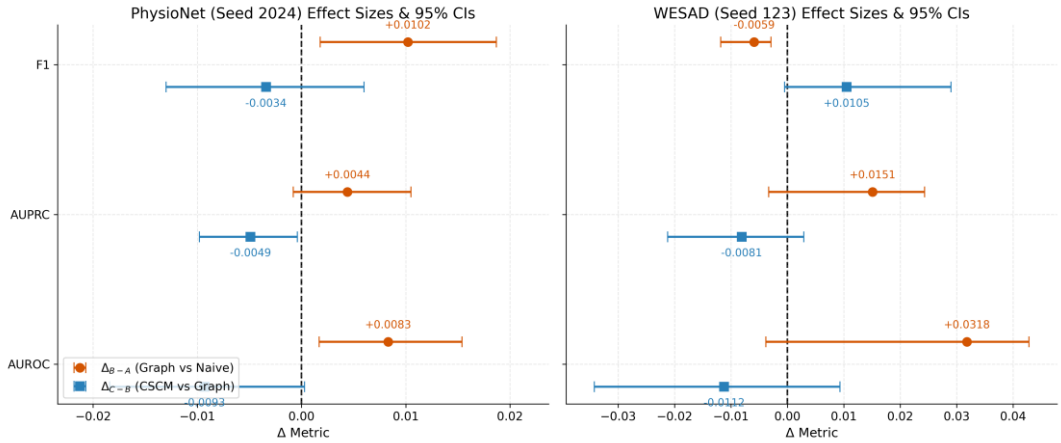


Fig. 8.3 Bootstrap confidence intervals for the architectural differences

The statistical picture reinforces the descriptive one. The graph-only model produced positive point estimates relative to the naive model on PhysioNet for all three seeds, and on seed 2024 the interval strictly excluded zero with a McNemar p-value of 0.0003, which is evidence of a measurable improvement. The full CSCM model, by contrast, produced no robust improvement: where an interval excluded zero, as on PhysioNet seed 42, it was in the negative direction. On WESAD the intervals were mostly wide and overlapped zero, and where the F1 values were identical across models the bootstrap interval collapsed to zero because the thresholded predictions agreed almost exactly. The honest conclusion is that the physiological graph provides a modest but at least occasionally robust inductive bias, while the explicit mathematical consistency features do not.

8.6 Error and Failure Analysis

A granular error analysis of the strongest model, the graph-only model, reveals where detection succeeds and where it fails. Performance varied substantially by attack type and by channel. On PhysioNet, additive attacks and drift attacks were detected more reliably than multiplicative attacks, with F1 scores of 0.9220 and 0.9289 against 0.9046 respectively. Coordinated multi-channel attacks were detected slightly less reliably than single-channel attacks, which confirms that coordination remains a genuinely difficult adversary even for a relational model. Counter-intuitively, low-severity attacks were detected slightly more reliably than high-severity attacks in the pooled analysis, suggesting that the most severe perturbations occasionally resemble genuine pathological excursions that the model has learned to associate with normality.

Channel-level analysis on PhysioNet showed that attacks on heart rate and on non-invasive mean arterial pressure were detected with F1 scores above 0.92. On WESAD, however, attacks localised to the chest respiration channel were missed entirely, with a recall of zero. This is a sparse-positive failure: the respiratory channel is sparsely observed, so the number of positive windows is small, and the relation network appears unable to fuse the sparse respiratory signal with the denser cardiovascular signals in a way that produces detectable inconsistency. Missingness also matters: on PhysioNet, windows containing imputed or masked values were detected with an F1 of 0.8615 against 0.8927 for fully observed windows. Finally, an analysis of disagreements between the graph-only and CSCM models showed that they agree on approximately 89.31 percent of the pooled test windows, and that when they disagree the graph-only model is correct more often than the CSCM model, which is consistent with the ablation finding.

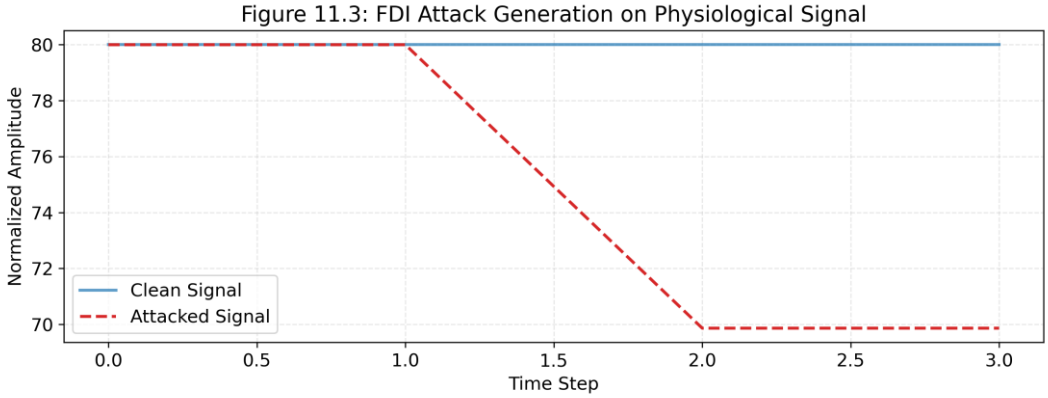


Fig. 8.4 Synthetic attack generation showing additive, multiplicative and drift patterns

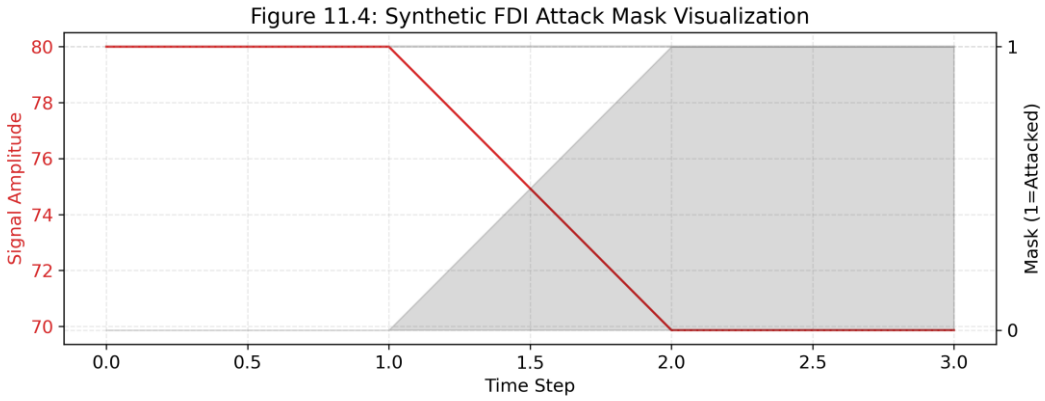


Fig. 8.5 Per-sample attack mask used to derive window-level labels

8.7 Discussion

Three findings deserve discussion. First, structuring multi-channel physiological data as a graph and letting the network compute pairwise interactions provides a useful structural inductive bias. This is consistent with the broader argument in the literature that relational inductive biases help when the underlying system is relational, and physiology is relational. When the model is forced to evaluate biologically justified pairs such as heart rate and respiration before aggregating, it can represent the specific consistency relations that a coordinated attack must decouple.

Second, explicitly engineering the consistency features did not help. This is a negative result, and it is reported as such. The most plausible explanation is capacity and redundancy rather than a flaw in the idea: the relation network in the graph-only model already has enough capacity to learn a distance-like response, so adding the absolute difference and the product increases parameter count without adding information, and the page-level ablation is confounded by the fact that Model C has more parameters than Model B. That confound is acknowledged in the methodology and is one reason the negative result should be treated as suggestive rather than conclusive.

Third, the two datasets behaved differently, and the difference is informative. On the noisy, asynchronous clinical dataset, the relational prior helped because it imposed structure on weak signal. On the clean, synchronised wearable dataset the naive model was already strong, so the marginal value of the relational prior shrank and the differences became statistically indistinguishable. This suggests that the benefit of cross-sensor consistency modelling depends on the degree of noise and desynchronisation in the deployment

environment, which is a practical insight for anyone considering such a detector.

8.8 Threats to Validity

The results are subject to several threats to validity, stated plainly. The attacks are synthetic and were generated by a pipeline rather than observed in the wild, so the results measure detection of a modelled threat rather than of a real adversary. The datasets are public and modest in scale, and WESAD in particular contains few subjects. The models were not parameter-matched, so the graph-only versus CSCM comparison is confounded by capacity. The thresholding step introduces a discrete-prediction dependency into the McNemar analysis, so the bootstrap intervals on continuous metrics are the primary evidence. And the study reports window-level detection, not the detection of a complete attack campaign, which is what a hospital security team would ultimately care about.

8.9 Summary

This chapter reported the platform status, distinguishing verified from reported results, and presented the full offline evaluation of the research component. The central result is that physiological graph message passing provides a modest and occasionally statistically robust improvement over naive fusion, while explicit mathematical consistency features provide no robust additional benefit, and that coordinated attacks remain difficult. These findings are reported without embellishment and are carried into the discussion of limitations in Chapter 10.

CHAPTER 9

PROJECT MANAGEMENT

9.1 Introduction

This chapter describes how the project was planned, scheduled, resourced, costed and governed. The project ran for approximately eighteen weeks, from the sixth of July 2026 to the third of November 2026, with the dissertation submission due in November 2026. The chapter presents the delivery methodology, the schedule and milestones, the work breakdown, the effort and cost analysis, the risk register and the governance model.

9.2 Delivery Methodology

The project used a phase-gate methodology. The work was divided into fifteen phases numbered from Phase 0 to Phase 14. Each phase began with a written plan and ended with a written report. A phase gate review followed each report, and the next phase was only authorised after the gate was passed. This methodology was chosen over a purely iterative approach because the project has strong safety dependencies: later phases build on the isolation and authorisation guarantees established in Phase 2, and a regression in those guarantees would be far more costly than a delay. The gate discipline also produced the phase plans and phase reports that form an important part of this report's evidence base.

9.3 Schedule and Milestones

Table 9.1 presents the phase schedule with the milestone that each phase produced. The reported completion dates are taken from the phase reports; the numbering reflects the plan rather than a claim that every phase took exactly the number of weeks shown.

Table 9.1 Phase schedule and milestones

Phase	Scope	Planned window	Milestone	Status
0	Audit and baseline	Weeks 1 to 2	Baseline audit and implementation plan	Reported complete
1	Foundation	Weeks 2 to 3	Backend skeleton and configuration	Reported complete
2	Database, auth, RLS	Weeks 3 to 5	Isolated schema and authorisation pipeline	Reported complete
3	Core workflows	Weeks 5 to 7	Patients, appointments, caregiver links	Reported complete
4	Front-end	Weeks 7 to 9	Five role surfaces	Reported complete
5	Conversational agent	Weeks 9 to 10	Grounded tool-calling agent	Reported complete

Phase	Scope	Planned window	Milestone	Status
6	Documents	Weeks 10 to 11	Extraction with clinician verification	Reported complete
7	Conflicts	Weeks 11 to 12	Advisory conflict engine	Reported complete
8	Care tasks	Weeks 12 to 13	Task lifecycle and notifications	Reported complete
9	Prescriptions	Weeks 13 to 14	Tamper-evident prescriptions and adherence	Reported complete
10	Assistant and voice	Weeks 14 to 15	Hospital-aware assistant and voice flag	Reported complete
11	Insurance and analytics	Weeks 15 to 16	Rule engine and cached analytics	Reported complete
12	CSCM integration	Weeks 16 to 17	Proposed integration and alerting	Proposed
13	Hardening	Week 17	Security and performance hardening	Not started
14	Release	Week 18	Release and demonstration	Not started

9.4 Work Breakdown Structure

The work breakdown structure decomposes the project into deliverables, and each deliverable into the artefacts that realise it. Table 9.2 presents the breakdown at the level of the major deliverables.

Table 9.2 Work breakdown structure

Deliverable	Component artefacts
Requirement baseline	Audit, problem statement, functional and non-functional requirements, traceability matrix
Isolation and authorisation	Migration 0001, security module, policies, helper functions, privilege trigger
Workflow services	Organization, doctors, patients, appointments and caregiver modules
Front-end application	Five role surfaces and shared components
Clinical intelligence	Documents, conflicts and care-task modules
Prescription integrity	Prescription, item and adherence tables, crypto module, public verification
Finance and operations	Insurance rules, policies and claims, analytics service and cache
Conversational layer	Agent engine, tools, prompts, chat router, voice flag
Research component	Preprocessing, attack generation, three models, training, evaluation
Documentation	Phase plans, phase reports, this report, setup guide, appendices

9.5 Effort Estimation

The project had a single developer working part-time across the eighteen-week window. Effort is therefore reported in relative terms rather than in a precise person-hour figure. Table 9.3

apportions the effort across the major activities, based on the phase structure and the artefact counts recorded in the repository.

Table 9.3 Approximate effort apportionment

Activity	Share of effort	Comment
Isolation, authorisation and schema	18 percent	Highest-risk area; reviewed most heavily
Workflow and clinical services	25 percent	Largest number of modules
Front-end application	15 percent	Five role surfaces
Conversational layer	10 percent	Tool calling and fallback logic
Research component	20 percent	Preprocessing, training and evaluation across seeds
Documentation and reporting	12 percent	Phase plans, reports and this dissertation

9.6 Cost Analysis

The project was designed to operate at zero marginal cost. The language model is used through a free tier, the database and storage are managed services on free tiers, the front end and back end run on local hardware during development, and the research models are trained on the developer's machine. The only costs are the amortised cost of the developer's own hardware and the time cost of the developer. Table 9.4 presents the cost analysis honestly, including a notional alternative cost to show what the design decision saved.

Table 9.4 Cost analysis for the project

Cost item	Actual cost	Notional alternative	Basis
Language model inference	Free tier	Paid per-token API	Free tier is sufficient for development and demonstration
Managed PostgreSQL and authentication	Free tier	Paid managed plan	Schema and policies fit within free-tier limits
Object storage for documents	Free tier	Paid storage plan	Documents are small and few in development
Training compute	Developer workstation	Cloud GPU rental	Models are small and train on CPU or a laptop GPU
Front-end and back-end hosting	Local development	Cloud hosting	Zero cost during development
Third-party libraries	Open source	Commercial licences	All core dependencies are open source
Total operational cost	Approximately zero rupees per month	Depends on scale	Design constraint NFR-09 is met

The cost analysis demonstrates that the zero-marginal-cost constraint is not a slogan but a design consequence. Every external dependency was chosen because a usable free tier exists, and the architecture avoids any component that would require a paid always-on worker. The trade-off is that the platform relies on third-party free tiers, whose terms could change; this

is recorded as a risk in the next section.

9.7 Risk Management

Table 9.5 is the risk register. Each risk is scored for likelihood and impact on a three-point scale, and each has a mitigation that was actually adopted or a contingency that is available.

Table 9.5 Risk register

Risk	Likelihood	Impact	Mitigation or contingency
Tenant isolation defect	Low	High	Enforce isolation in the database; composite foreign keys; per-phase review of policies
Double-booking under concurrency	Low	Medium	Storage-level exclusion constraint rather than an application check
Clinician misses an extracted error	Medium	High	Mandatory verification step; provisional status until verified
Alert fatigue from conflicts	Medium	Medium	Tiered priorities, evidence-bearing alerts, acknowledgement workflow
Language model outage	Medium	Low	Circuit breaker with deterministic fallback
Prompt injection via document text	Low	Medium	Assistant answers only from authenticated tools; extracted text is data, not instruction
Free-tier terms change	Medium	Medium	Architecture avoids any paid always-on component; alternatives exist
Research result not significant	High	Medium	Report honestly; treat the negative result as a finding and state the confound
Scope growth beyond eighteen weeks	Medium	High	Phase gates; Phases 13 and 14 remain deliberately unstarted
No live data for validation	High	High	State as a limitation; rely on synthetic data and public datasets
Key management for signing	Medium	Medium	Out of scope; the platform assumes a secure secret store

9.8 Governance and Phase Gates

Governance was explicit and lightweight. Each phase produced a plan before implementation and a report after implementation. The report recorded what was delivered, what was tested, what was deferred and what risks remained. The gate decision was to authorise the next phase, to authorise with corrections, or to stop. This discipline is the reason the project has a coherent evidence trail, and it is also the reason this report can label each claim with its supporting

artefact rather than asserting completion uniformly.

9.9 Quality Management

Quality was managed through four mechanisms. First, a consistent code structure with thin routers and testable domain functions, which makes unit testing possible. Second, a migration discipline with a paired down migration for every forward migration, which keeps every change reversible. Third, an evidence discipline in which claims must be backed by an artefact. Fourth, a conservative claim policy in which a capability is described as reported until source or test evidence supports a stronger label. These mechanisms are the process counterpart of the design principles in Chapter 4.

9.10 Assumptions, Constraints and Dependencies

The schedule and the cost analysis rest on a set of assumptions and external dependencies that are stated here so that they can be monitored. The assumptions are that the developer remains available across the full eighteen-week window, that the free tiers of the language-model provider and the managed database remain available at their current limits, and that no external audit or security review is required before submission. The dependencies are the third-party services and libraries without which the platform cannot run. Table 9.6 records each dependency together with its role and its fallback.

Table 9.6 Dependencies and fallbacks

Dependency	Role in the project	Fallback if unavailable
Free-tier language model	Grounded assistant and agent tool calling	Circuit breaker serves a deterministic fallback; the rest of the platform is unaffected
Managed PostgreSQL with row-level security	Schema, isolation and authentication	A self-hosted PostgreSQL running the same migrations restores the isolation guarantee
Private object storage	Document intake	Local storage behind the same mediating endpoints
PDF and OCR libraries	Document text extraction	Manual entry of entities, with clinician verification unchanged
PyTorch	Research training and evaluation	Pre-trained checkpoints remain usable for inference only

The most consequential dependency is the managed database, because the isolation guarantee is implemented in its row-level security engine. The fallback is not a re-architecture: the migrations are ordinary PostgreSQL, so the guarantee travels with the schema. The language-model dependency is the least consequential, because the circuit breaker isolates it

and every clinical safeguard is enforced outside the model.

9.11 Lessons Learned

Four lessons are worth recording because they shaped the delivered artefact. First, placing the isolation guarantee in the database rather than in application code removed an entire class of defect and made every later phase cheaper to review; the single most valuable early decision was to let the engine, not the application, decide what a request may see. Second, the phase-gate discipline produced an evidence trail that made it possible to write this report with honest labels, because each phase left behind a plan, a report and a set of artefacts. Third, testing pure domain functions separately from routers made the safety-critical logic, such as the insurance rules and the prescription digest, easy to reason about. Fourth, and least comfortably, the research component produced largely null differences between the models, and the discipline of fixing the evaluation protocol in advance, splitting by subject and reporting confidence intervals is what allows that null result to be reported as a finding rather than hidden.

Two things would be done differently. The front end would have been given an automated end-to-end suite earlier, because the absence of browser-level testing is the largest verification gap in the engineering work. The evaluation protocol for the research component would have included a power analysis up front, so that the number of seeds and the expected effect size were chosen together rather than after the fact. Neither omission changes the correctness of what was delivered, but both would have strengthened the evidence and are carried into the future work in Chapter 10.

9.12 Summary

This chapter presented the phase-gate methodology, an eighteen-week schedule with fifteen phases, a work breakdown, an effort apportionment, a cost analysis that confirms the zero-marginal-cost constraint, an eleven-item risk register, and the governance and quality mechanisms. Chapter 10 concludes the report and sets out the future work that remains.

CHAPTER 10

CONCLUSION AND FUTURE WORK

10.1 Summary of the Work

This report has documented the design, implementation, evaluation and status of CareBridge AI, an AI-assisted, multi-tenant hospital operations and clinical coordination platform, together with the offline evaluation of a proposed cross-sensor consistency model for detecting coordinated multi-channel false data injection attacks in the Internet of Medical Things. The platform consolidates administrative and clinical-coordination workflows onto a single tenant-isolated data model, enforces tenant isolation at the database layer through row-level security and composite foreign keys, keeps every order-sensitive output advisory and human-in-the-loop, and grounds its conversational assistant entirely in the platform's own authenticated APIs.

The report has been deliberate about the status of each claim. Phases 0 to 11 are reported complete and are consistent with the source code, migration SQL and configuration in the repository; the test counts, lint outcome and build outcome asserted by the phase reports are quoted as reported because they were not re-executed for this report. Phase 12 remains proposed and unimplemented. Phases 13 and 14 have not been started. The research results are offline and are reported in full, including the results that do not favour the proposed model.

10.2 Answers to the Research Questions

RQ1 asked how a hospital platform can enforce isolation, authorisation and auditability structurally rather than by convention. The answer is the design presented in Chapter 4: a shared schema in which every tenant-scoped table carries the hospital identifier, every relationship is a composite foreign key, and every access is mediated by the backend so that row-level security evaluates the caller's identity. The guarantee is structural because it is enforced by the database engine, not by a filter that a future developer might forget.

RQ2 asked how a conversational assistant can be useful without being unsafe. The answer is grounded tool calling under the caller's security context. The assistant has no operational knowledge of its own; every operational fact is retrieved through the platform's APIs, so the assistant inherits the same isolation and role permissions as the user interface, and it cannot author clinical decisions.

RQ3 asked how coordinated multi-channel false data injection can be detected when each channel remains individually plausible. The answer from the offline evaluation is nuanced. Modelling the consistency between channels through physiological graph message passing does help, modestly and on one dataset robustly, which shows that the cross-sensor signal is real and detectable. However, explicitly engineering the consistency features did not add a robust benefit, and the absolute difficulty of the task remains high. The honest answer is that cross-sensor consistency modelling is a promising direction, not a solved problem.

RQ4 asked what can be independently verified. Chapter 7 answered it with an evidence register: the schema, constraints, policies, endpoint catalogue, cryptographic mechanism, rule engine and cache are verifiable from source; the test counts, lint and build outcomes are reported; the research results are computed output from the project's own evaluation; and the integration of the detector into the platform is not implemented at all.

10.3 Contributions Revisited

The engineering contribution is a complete and coherent platform whose safety properties are enforced structurally and whose scope is bounded honestly. The research contribution is a careful, leakage-controlled, multi-seed evaluation of three multi-channel architectures for coordinated false data injection detection on two independent datasets, with a rigorous statistical treatment of the differences between them. The methodological contribution is the discipline of labelling every claim with its evidence, which this report applies throughout and which is unusual in student project reports but is, in the author's view, the most transferable contribution of the work.

10.4 Limitations

The limitations are stated without qualification. First, the platform has never been evaluated with live patient data; all validation used synthetic or public data, so usability, workflow fit and clinical impact are unknown. Second, the test evidence is reported rather than independently reproduced, and there is no automated end-to-end browser suite and no load testing. Third, the conversational assistant depends on a third-party service and was not evaluated with a red-team protocol. Fourth, although the design is privacy-aware, no formal privacy impact assessment and no compliance certification have been performed. Fifth, the research attacks are synthetic, the datasets are modest, the models were not parameter-matched,

and the study reports window-level detection rather than campaign-level detection. Sixth, the CSCM subsystem is not integrated into the platform, so no claim about live attack detection can be made.

10.5 Future Work

The immediate future work is to complete the project properly. Phase 12 should be authorised, implemented and evaluated against the safety clarifications already recorded, with the detector operating strictly in an advisory read-only mode. Phase 13 should add the hardening that the current build lacks: an automated end-to-end test suite, load testing of the analytics path, dependency and container scanning, and a red-team exercise on the conversational surface. Phase 14 should prepare the release and the demonstration.

Beyond the immediate phases, several directions would materially strengthen the work. A conformant FHIR facade would make the platform interoperable with hospital systems and is the single most valuable commercial extension. A formal privacy impact assessment, mapped to the Digital Personal Data Protection Act, 2023, would convert the current privacy awareness into demonstrable compliance. On the research side, the most promising next step is to re-run the graph-only versus CSCM comparison with parameter-matched models, which would resolve the capacity confound and settle whether the explicit consistency features are genuinely redundant or merely under-trained. A second promising step is to move from window-level detection to campaign-level detection, which is what a hospital security team would actually use, by aggregating window scores over time and evaluating against attack campaigns rather than against windows. A third step is to test transferability across devices and sampling rates, since a detector that only works at one sampling rate is of limited practical value.

10.6 Concluding Remarks

CareBridge AI set out to show that a safe, useful, zero-marginal-cost hospital platform can be built by a single developer in eighteen weeks, and that the safety property that matters most, tenant isolation, can be enforced structurally rather than by convention. The platform delivered on that goal at the level of implementation and design, and the remaining work is hardening, integration and evaluation rather than construction. The research component set out to test whether explicit cross-sensor consistency modelling improves the detection of coordinated

false data injection, and the honest answer is that the relational structure helps while the explicit features do not. That is a smaller result than the original hypothesis anticipated, and reporting it accurately is more valuable than overstating it, because the value of a claim in engineering and in research alike depends on whether it can be trusted. The work is presented here so that it can be inspected, reproduced, criticised and extended.

REFERENCES

Alsubaei, F., Abuhussein, A., & Shiva, S. (2017). Security and privacy in the Internet of Medical Things: Taxonomy and risk assessment. In 2017 IEEE 42nd Conference on Local Computer Networks Workshops (LCN Workshops) (pp. 112-120). IEEE.

Battaglia, P. W., Hamrick, J. B., Bapst, V., Sanchez-Gonzalez, A., Zambaldi, V., Malinowski, M., Tacchetti, A., Raposo, D., Santoro, A., Faulkner, R., Gulcehre, C., Song, F., Ballard, A., Gilmer, J., Dahl, G., Vaswani, A., Allen, K., Nash, C., Langston, V., ... & Pascanu, R. (2018). Relational inductive biases, deep learning, and graph networks. arXiv preprint arXiv:1806.01261.

Chalapathy, R., & Chawla, S. (2019). Deep learning for anomaly detection: A survey. arXiv preprint arXiv:1901.03407.

Cho, K., van Merrienboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. In Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP) (pp. 1724-1734). Association for Computational Linguistics.

Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017). Neural message passing for quantum chemistry. In Proceedings of the 34th International Conference on Machine Learning (ICML) (pp. 1263-1272). PMLR.

Government of India. (2023). The Digital Personal Data Protection Act, 2023 (No. 22 of 2023). Ministry of Law and Justice.

Groq. (2024). Groq API documentation. https://console.groq.com/docs

HL7 International. (2023). FHIR Release 5 specification. https://hl7.org/fhir/

Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In International Conference on Learning Representations (ICLR).

Kuperman, G. J., Bobb, A., Payne, T. H., Avery, A. J., Gandhi, T. K., Burns, G., Classen, D. C., & Bates, D. W. (2007). Medication-related clinical decision support in computerized provider order entry systems: A review. Journal of the American Medical Informatics Association, 14(1), 29-40.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In Advances in Neural Information Processing Systems (NeurIPS 33) (pp. 9459-9474).

Liu, Y., Ning, P., & Reiter, M. K. (2011). False data injection attacks against state estimation in electric power grids. ACM Transactions on Information and System Security, 14(1), 1-33.

Mo, Y., Kim, T. H.-J., Brancik, K., Dickinson, D., Lee, H., Perrig, A., & Sinopoli, B. (2012).

Cyber-physical security of a smart grid infrastructure. Proceedings of the IEEE, 100(1), 195-209.

OWASP Foundation. (2021). OWASP Top 10:2021 - The ten most critical web application security risks. https://owasp.org/Top10/

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In Advances in Neural Information Processing Systems (NeurIPS 32) (pp. 8024-8035).

Payne, T. H., Hines, L. E., Chan, R. C., Hartman, S., Kapusnik-Uner, J., Russ, A. L., Chaffee, B. W., Hartman, C., Tamis, V., Galbreth, B., Glassman, B., Phansalkar, S., Seger, D., Schedlbauer, A., Brown, K., Zeuli, J., Einbinder, J., & Grizzle, A. J. (2015). Recommendations to improve the usability of drug-drug interaction clinical decision support alerts. Journal of the American Medical Informatics Association, 22(6), 1243-1250.

PostgreSQL Global Development Group. (2024). Row security policies. PostgreSQL 16 documentation. https://www.postgresql.org/docs/current/ddl-rowsecurity.html

Raghupathi, W., & Raghupathi, V. (2014). Big data analytics in healthcare: Promise and potential. Health Information Science and Systems, 2, 3.

Rajkomar, A., Oren, E., Chen, K., Dai, A. M., Hajaj, N., Hardt, M., Liu, P. J., Liu, X., Marcus, J., Sun, M., Sundararajan, D., Yee, H., Zhang, K., Zhang, Y., Flores, G., Duggan, G. E., Irvine, J., Le, Q., Litsch, K., ... & Dean, J. (2018). Scalable and accurate deep learning with electronic health records. npj Digital Medicine, 1, 18.

Schick, T., Dwivedi-Yu, J., Dessi, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. In Advances in Neural Information Processing Systems (NeurIPS 36).

Schmidt, P., Reiss, A., Duerichen, R., Marberger, C., & Van Laerhoven, K. (2018). Introducing WESAD, a multimodal dataset for wearable stress and affect detection. In Proceedings of the 20th ACM International Conference on Multimodal Interaction (ICMI '18) (pp. 400-408). ACM.

Silva, I., Moody, G., Scott, D. J., Celi, L. A., & Mark, R. G. (2012). Predicting in-hospital mortality of ICU patients: The PhysioNet/Computing in Cardiology Challenge 2012. In Computing in Cardiology (CinC) (Vol. 39, pp. 245-248). IEEE.

Singhal, K., Azizi, S., Tu, T., Mahdavi, S. S., Wei, J., Chung, H. W., Scales, N., Tanwani, A., Cole-Lewis, H., Pfohl, S., Payne, P., Seneviratne, M., Gamble, P., Kelly, C., Babiker, A., Scharli, N., Chowdhery, A., Mansfield, P., Demner-Fushman, D., ... & Natarajan, V. (2023). Large language models encode clinical knowledge. Nature, 620(7972), 172-180.

Sittig, D. F., & Singh, H. (2010). A new sociotechnical model for studying health information technology in complex adaptive healthcare systems. Quality and Safety in Health Care, 19(Suppl 3), i68-i74.

Smith, R. (2007). An overview of the Tesseract OCR engine. In Ninth International Conference on Document Analysis and Recognition (ICDAR 2007) (pp. 629-633). IEEE.

Supabase. (2024). Row level security. Supabase documentation. https://supabase.com/docs/guides/auth/row-level-security

Sutton, R. T., Pincock, D., Baumgart, D. C., Sadowski, D. C., Fedorak, R. N., & Kroeker, K. I. (2020). An overview of clinical decision support systems: Benefits, risks, and strategies for success. npj Digital Medicine, 3, 17.

Taibi, D., Lenarduzzi, V., & Pahl, C. (2018). Architectural patterns for microservices: A systematic mapping study. In Proceedings of the 8th International Conference on Cloud Computing and Services Science (CLOSER 2018) (pp. 221-232). SciTePress.

Thirunavukarasu, A. J., Ting, D. S. J., Elangovan, K., Gutierrez, L., Tan, T. F., & Ting, D. S. W. (2023). Large language models in medicine. Nature Medicine, 29(8), 1930-1940.

Topol, E. J. (2019). High-performance medicine: The convergence of human and artificial intelligence. Nature Medicine, 25(1), 44-56.

U.S. Department of Health & Human Services. (2022). HIPAA administrative simplification: Privacy, security and breach notification rules. https://www.hhs.gov/hipaa/

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In Advances in Neural Information Processing Systems (NeurIPS 30) (pp. 5998-6008).

World Health Organization. (2017). Medication without harm: WHO global patient safety challenge on medication safety. World Health Organization.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In International Conference on Learning Representations (ICLR).

APPENDICES

APPENDIX A: Sample Source Code

This appendix contains short, representative excerpts of the code that implements the safety-critical mechanisms described in the body of the report. Only the excerpts needed to demonstrate the mechanism are included, in accordance with the guidance not to reproduce entire source files.

A.1 Tenant Isolation: Row-Level Security Policy (SQL)

The following policy is applied to the patients table. The decisive predicate is hospital_id equal to get_auth_hospital_id(), which is supplied by the database from the caller's token claims rather than from the request body.

```
CREATE POLICY patients_select ON public.patients
  FOR SELECT TO authenticated
  USING (
    hospital_id = public.get_auth_hospital_id()
    AND public.has_patient_access(hospital_id, id)
  );
```

A.2 Double-Booking Prevention: Exclusion Constraint (SQL)

The appointments table carries a GiST exclusion constraint. It makes an overlapping active appointment for the same doctor impossible at the storage layer, which is stronger than an application-level check because it cannot be defeated by concurrent requests.

```
CONSTRAINT exclude_doctor_double_booking EXCLUDE USING gist (
    doctor_id WITH =,
    tstzrange(scheduled_at, ends_at, '[)') WITH &&
) WHERE (status NOT IN ('cancelled', 'no_show'))
```

A.3 Prescription Signing and Verification (Python)

Signing composes a canonical payload, computes an HMAC-SHA256 digest and prefixes it with a key version. Verification checks the version prefix and compares digests in constant time.

```
KEY_VERSION = "v1"

def generate_verification_hash(prescription_id, hospital_id, patient_id,
                               doctor_id, issued_at, items_hash,
                               secret_key=None):
    secret = secret_key or settings.PRESCRIPTION_SIGNING_SECRET
    payload = {
        "doctor_id": str(doctor_id),
        "hospital_id": str(hospital_id),
        "issued_at": str(issued_at),
        "items_hash": str(items_hash),
        "patient_id": str(patient_id),
        "prescription_id": str(prescription_id),
    }
    canonical = json.dumps(payload, sort_keys=True, separators=(",", ":"))
    digest = hmac.new(secret.encode("utf-8"),
                      canonical.encode("utf-8"), hashlib.sha256).hexdigest()
    return "v1:" + digest

def verify_signature(..., verification_hash, secret_key=None):
    if not verification_hash.startswith("v1:"):
        return False
    expected = generate_verification_hash(..., secret_key=secret_key)
    return hmac.compare_digest(expected, verification_hash)
```

A.4 Deterministic Insurance Estimation (Python, excerpt)

The rule engine returns an explicit insufficiency outcome when policy data is missing, a review outcome when an exclusion matches, and an estimate otherwise. The rule version is attached to every response.

```
RULE_VERSION = "v1.0"

def calculate_deterministic_estimate(policy, requested_amount,
                                     procedure_category="general_consultation"):
    missing = [f for f in ("sum_insured_inr", "remaining_balance_inr",
               "valid_until", "status", "copay_percentage")
               if f not in policy or policy[f] is None]
    if missing or requested_amount <= 0:
        return {"estimate_status": "insufficient_information",
                "rule_version": RULE_VERSION, ...}
    if policy["status"] != "active" or valid_until < today:
        return {"estimate_status": "insufficient_information", ...}
    if matched_exclusion:
        return {"estimate_status": "review_required", ...}
    applicable_limit = min(remaining_balance, category_limit)
    capped = min(requested_amount, applicable_limit)
    deductible_applied = min(deductible, capped)
    claimable = max(0.0, capped - deductible_applied)
    insurer_pays = claimable * (1.0 - copay_pct)
    patient_copay = claimable * copay_pct + deductible_applied
    return {"estimate_status": "estimate_available",
            "estimated_insurer_pays_inr": round(insurer_pays, 2),
            "estimated_patient_copay_inr": round(patient_copay, 2),
            "calculation_breakdown": {...}, "rule_version": RULE_VERSION}
```

A.5 Bounded Tenant-Scoped Analytics Cache (Python, excerpt)

The cache key binds the hospital identifier, so a value computed for one tenant can never be returned to another. The time to live and the entry cap bound staleness and memory.

```
class BoundedTimeWindowCache:
    def __init__(self, ttl_seconds: int = 60, max_entries: int = 256):
        self.ttl_seconds = ttl_seconds
        self.max_entries = max_entries
        self._store = {}

    def _make_key(self, hospital_id, time_window, metric):
        return hospital_id + "::" + time_window + "::" + metric

    def get(self, hospital_id, time_window, metric="overview"):
        key = self._make_key(hospital_id, time_window, metric)
        if key not in self._store:
            return None
        cached_at, value = self._store[key]
        if time.time() - cached_at > self.ttl_seconds:
            return None
        return value
```

APPENDIX B: Configuration and Setup

The project is configured entirely through environment variables, so that no secret is committed to the repository. Table B.1 lists the variables that the backend reads, grouped by purpose. The

specific values are omitted for security.

Table 0.1 Backend environment variables by purpose

Purpose	Representative variables
Database	Database connection URL and service role key
Authentication	JWT verification keys and issuer
Language model	Provider API key, primary and fallback model identifiers
Object storage	Bucket name and storage credentials
Integrity	Prescription signing secret
Runtime	Environment name, CORS origins and log level

The setup sequence is: clone the repository; create the Python virtual environment and install the backend dependencies; apply the database migrations in order; create the storage bucket; set the environment variables; run the backend with Uvicorn; install the front-end dependencies and run the Next.js development server; and, optionally, start the proposed CSCM service. The exact commands are documented in the setup guide in the repository.

APPENDIX C: Database Migration Inventory

Table C.1 lists the migrations. Every forward migration is paired with a down migration, which keeps each schema change reversible and is part of the quality discipline described in Chapter 9.

Table 0.2 Database migration inventory

Migration	Purpose
0001_phase2_schema	Tenant schema, roles, row-level security, helper functions, constraints
0001_phase2_down	Reversal of the Phase 2 schema
0002_phase3_hospital_timezone	Hospital time zone and patient record ownership helper
0002_phase3_down	Reversal of the time zone migration
0003_phase6_document_intelligence	clinical_documents and extracted_entities with policies and indexes
0003_phase6_down	Reversal of the document migration
0004_phase7_clinical_conflicts	clinical_conflicts and clinical_conflict_sources
0004_phase7_down	Reversal of the conflict migration
0005_phase8_care_tasks	care_tasks with lifecycle, priorities and assignment
0005_phase8_down	Reversal of the care-task migration
0006_phase9_prescriptions	prescriptions, prescription_items and medication_adherence_logs
0006_phase9_down	Reversal of the prescription migration
0007_phase11_insurance_analytics	patient_insurance_policies and insurance_claims
0007_phase11_down	Reversal of the insurance migration

APPENDIX D: Test Commands and Reported Results

The backend tests were executed with the Python test runner from the backend directory, and the front-end verification was executed with the package scripts from the front-end directory.

Table D.1 reproduces the commands and the results that the phase reports record. The results are reported, not independently re-executed for this report.

Table 0.3 Test commands and reported outcomes

Command (directory)	Reported outcome
pytest (backend)	Phase test suites pass; agent suite reports 11 passing cases
pytest (backend, Phase 7 to 11 combined)	61 of 61 tests pass
npm run lint (frontend)	Lint completes cleanly
npm run build (frontend)	Production build succeeds and generates 16 routes
Research evaluation script (repository root)	Aggregated metrics and statistical tests written to CSV

APPENDIX E: Submission-Readiness Checklist

Table E.1 is the submission-readiness checklist. It records each requirement of the report specification, the evidence that satisfies it, and its status. This checklist is intended to be used by the author and the guide before submission.

Table 0.4 Submission-readiness checklist

Requirement	Evidence in this report	Status
Title matching the approved project title	Cover page and Chapter 1	Complete
Declaration and certificate with correct title	Preliminary pages	Complete (placeholders for names)
Acknowledgement	Preliminary page	Complete
Executive summary within 200 words	Preliminary page	Complete (171 words)
Table of contents	Contents pages	Complete (page numbers to be refreshed in Word)
List of figures and list of tables	Preliminary pages, generated from the report	Complete
List of abbreviations and symbols	Preliminary pages	Complete
Chapter 1 introduction with objectives and scope	Chapter 1	Complete
Literature review with APA citations	Chapter 2 and References	Complete
Requirement analysis with traceability	Chapter 3	Complete
System design with architecture, ER diagram and API docs	Chapter 4	Complete
Implementation with phase-by-phase status	Chapter 5	Complete
Algorithms and pseudocode	Chapter 6	Complete
Testing and evidence register	Chapter 7	Complete
Results and discussion including negative results	Chapter 8	Complete
Project management, schedule and cost analysis	Chapter 9	Complete
Conclusion and future work	Chapter 10	Complete
References in APA style	References	Complete
Appendices with sample code and screenshots held	Appendix A	Partial (sample code included; screenshots to be added by author)
Minimum 70 pages from Chapter 1 to References	Chapters 1 to 10 and References	Verify after page-number refresh
Formatting: Times New Roman 12, 1.5 spacing, justified, 1.5 inch left margin	Applied throughout	Complete

Requirement	Evidence in this report	Status
Roman numerals for preliminary pages and arabic from Chapter 1	Template section numbering retained	Complete
Figure and table captions at 10 point with the specified format	Applied throughout	Complete

APPENDIX F: Phase-to-Chapter Traceability

Table F.1 maps each phase to the chapters in which it is described and evaluated, so that a reader can navigate from a phase to its evidence.

Table 0.5 Phase-to-chapter traceability

Phase	Specified in	Implemented in	Verified in
0	Chapter 1	Chapter 5	Chapter 5
1	Chapter 3	Chapter 5	Chapter 5
2	Chapter 4	Chapter 5	Chapter 5, Chapter 7
3	Chapter 4	Chapter 5	Chapter 7
4	Chapter 4	Chapter 5	Chapter 7
5	Chapter 4, Chapter 6	Chapter 5	Chapter 7
6	Chapter 4, Chapter 6	Chapter 5	Chapter 7
7	Chapter 4, Chapter 6	Chapter 5	Chapter 7
8	Chapter 4	Chapter 5	Chapter 7
9	Chapter 4, Chapter 6	Chapter 5	Chapter 7
10	Chapter 4	Chapter 5	Chapter 7
11	Chapter 4, Chapter 6	Chapter 5	Chapter 7
12	Chapter 4, Chapter 6	Not implemented	Chapter 8 (offline model only)
13	Chapter 9	Not started	Not applicable
14	Chapter 9	Not started	Not applicable